Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu*[1] and Tri Dao*[2]

[1]Machine Learning Department, Carnegie Mellon University

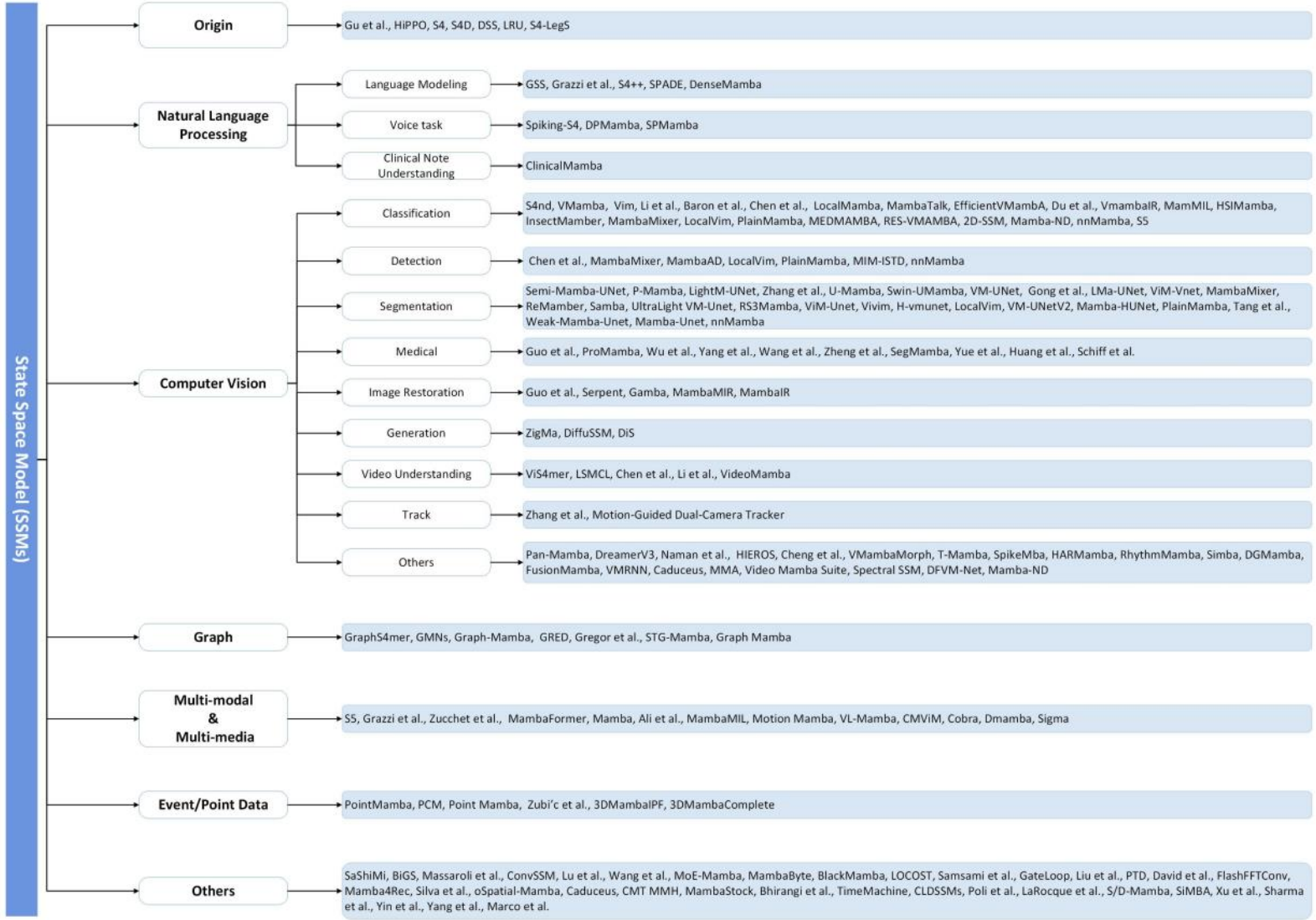
[2]Department of Computer Science, Princeton University
agu@cs.cmu.edu, tri@tridao.me

HAI
Human-centered AI

Table of Contents

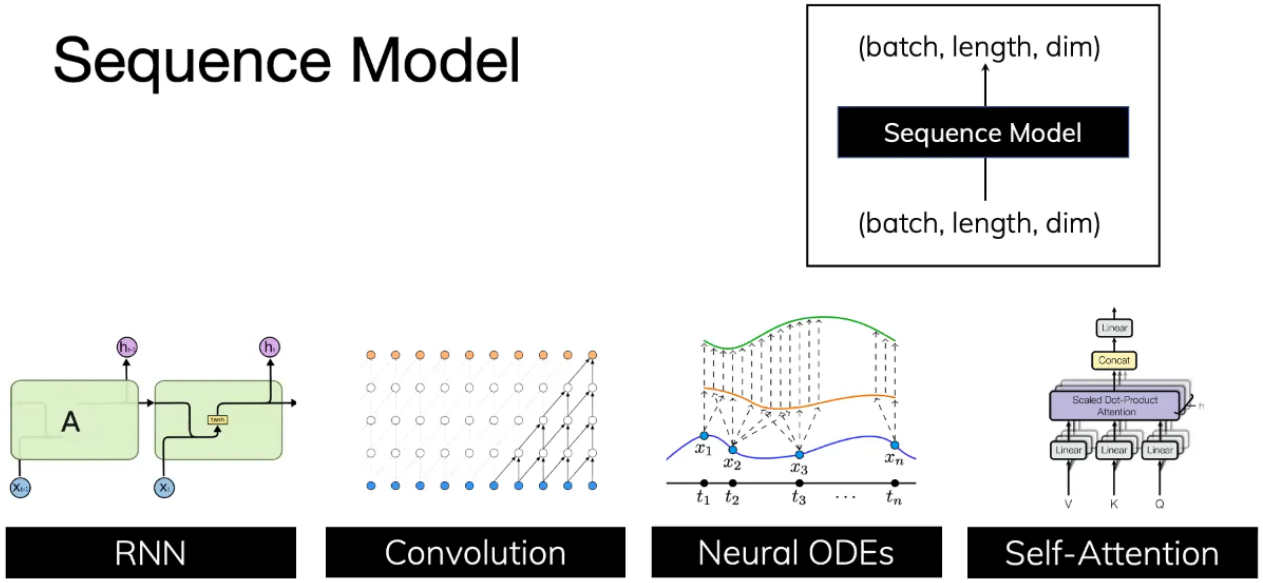
Preliminary

- What is Mamba?
  - Dealing with Sequential Data
    - Given $X_{1:T}$, predict X_{T+1}
  - Selective State Space Model
    - Without Attention
  - Now growing its own Ecosystem



Preliminary

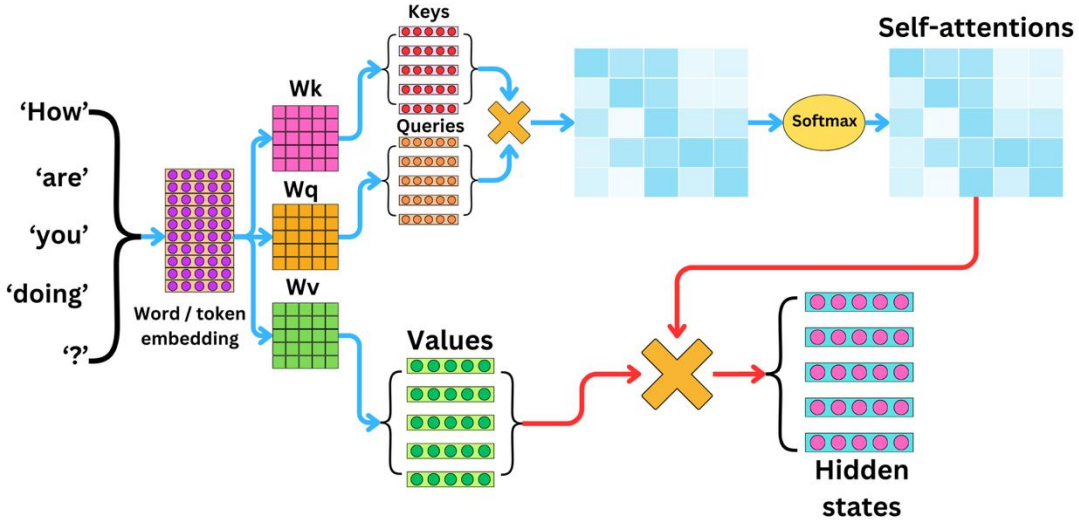
- Sequence Models
  - Analyses and predicts patterns in data over time.
    - e.g. Speech recognition, financial time series analysis, biosignal analysis



https://velog.velcdn.com/images/euisuk-chung/post/9e365b14-efb4-434d-8abd-9fde6bf9dfd5/image.png

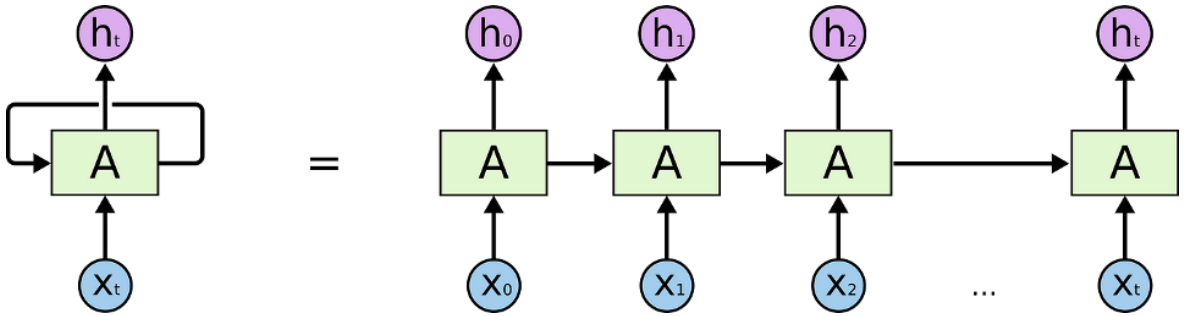
Preliminary

- Sequence Models
  - Transformer
    - Attention enables Training Parallelism → Fast Training
      - Compute all input tokens at the time
    - Attention forces Exhaustive Search → High Cost and Slow Inference
      - The time complexity of token generation: $O(n^2)$
      - If use KV cache: $O(n)$, but a plethora sequence length



Preliminary

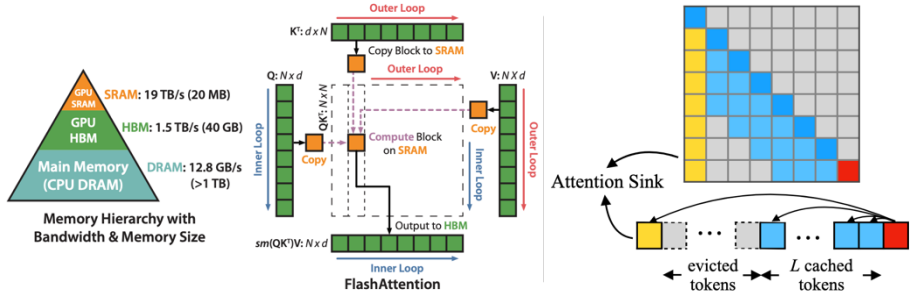
- Sequence Models
  - RNN
    - Memory's update cannot be Parallelised → Slow Training
      - The next time-step's memory must need before time-step's memory
    - Memory carry total information → Low Cost and Fast Inference
      - Current output only rely on previous memory

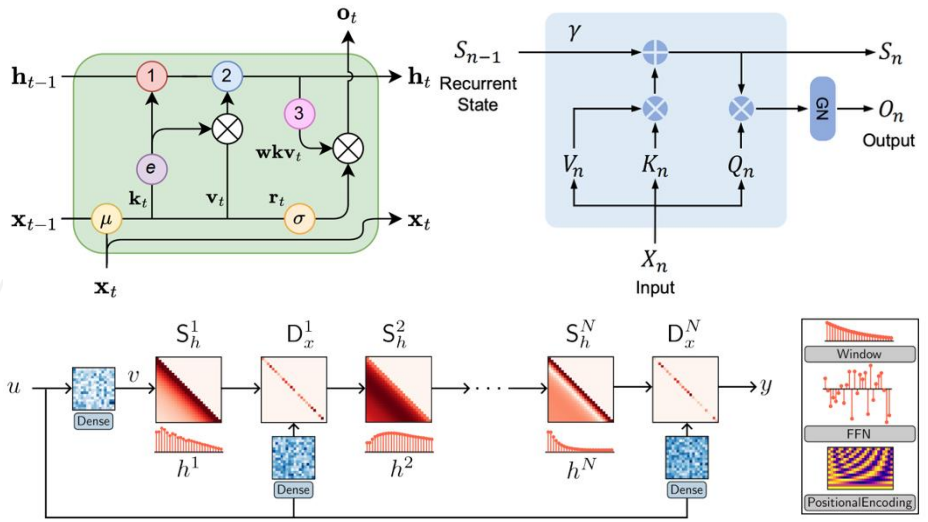


https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Preliminary

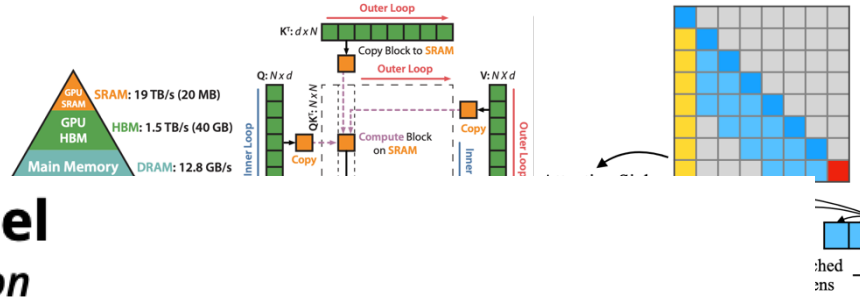
- Sequence Models
  - Transformer ⇒ towards Low Cost
    - Attention enables Training Parallelism → Fast Training
      - Compute all input tokens at the time
    - Attention forces Exhaustive Search → High Cost and Slow Inference
      - The time complexity of token generation: $O(n^2)$
      - If use KV cache: $O(n)$, but a plethora sequence length
  - RNN ⇒ towards Parallel Training and Performance
    - Memory's update cannot be Parallelised → Slow Training
      - The next time-step's memory must need before time-step's memory
    - Memory carry total information → Low Cost and Fast Inference
      - Current output only rely on previous memory

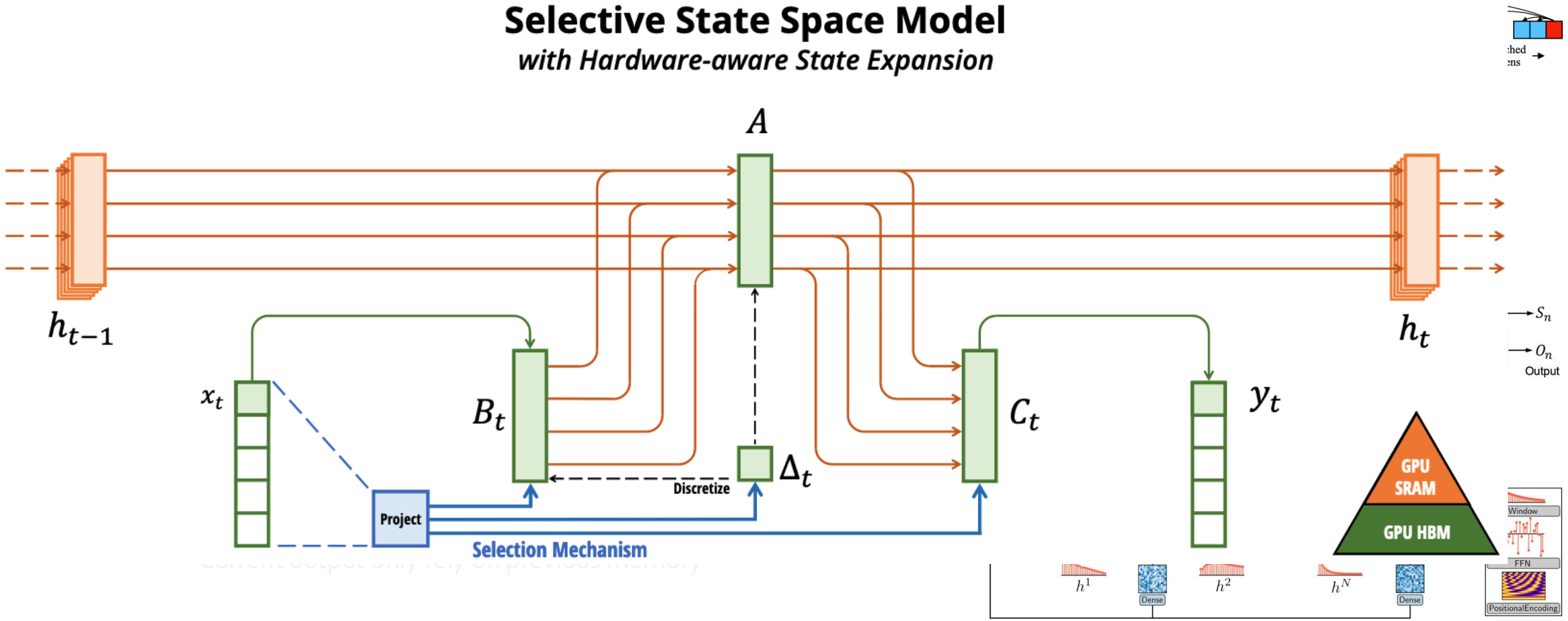




Preliminary

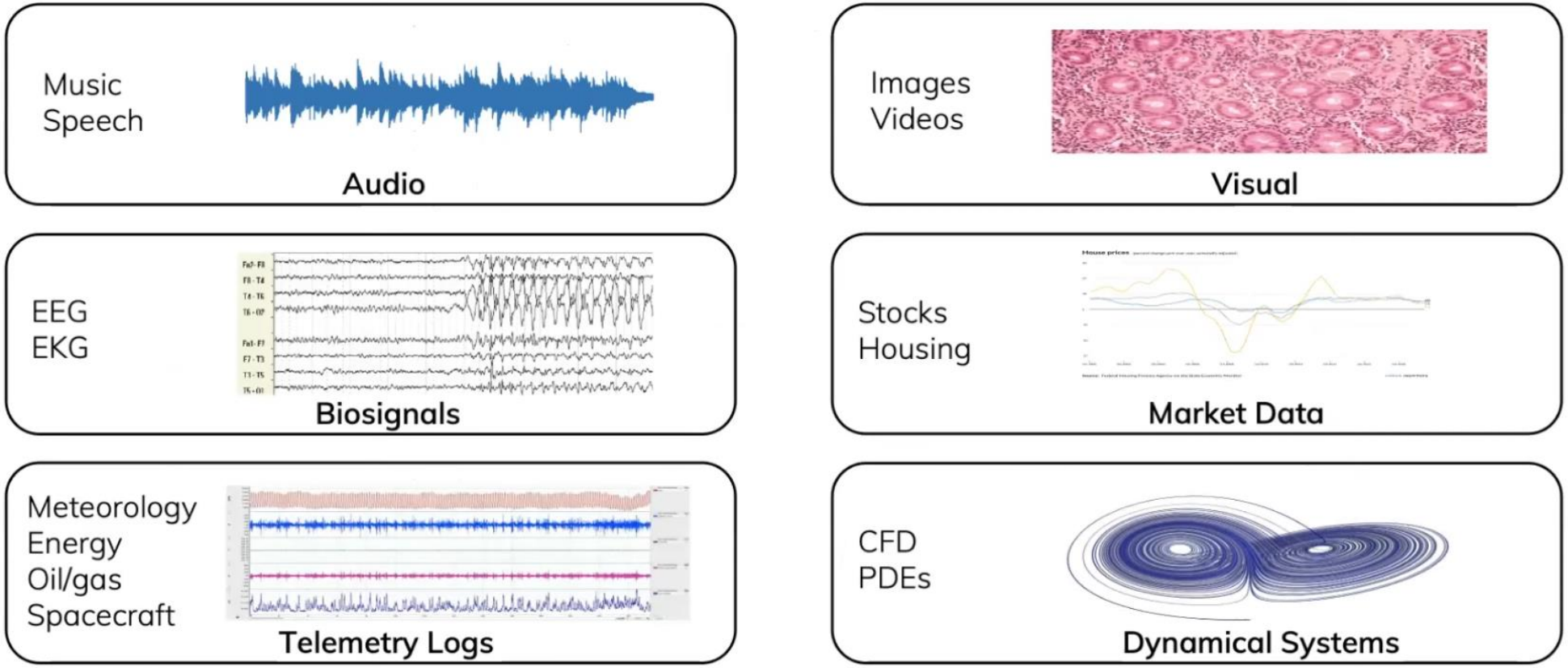
- Sequence Models





Preliminary

- Challenging Data for Current Sequence Model: Signal



Preliminary

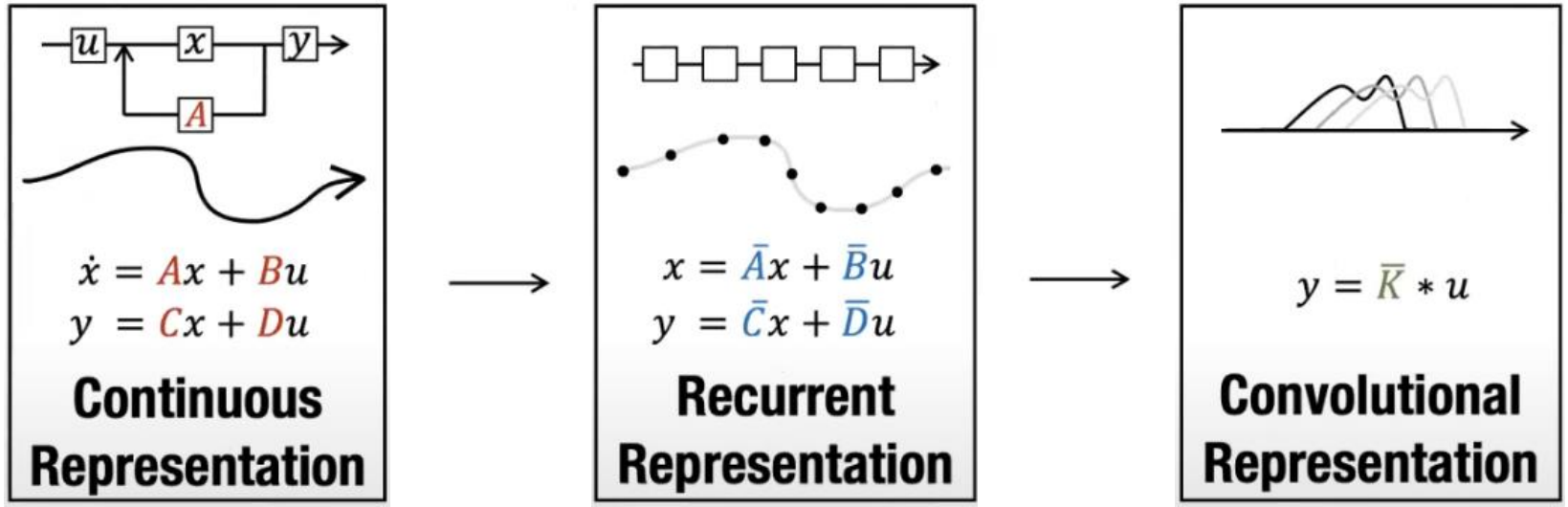
- Challenging Data for Current Sequence Model: Signal

100-1000

10000-1000000+

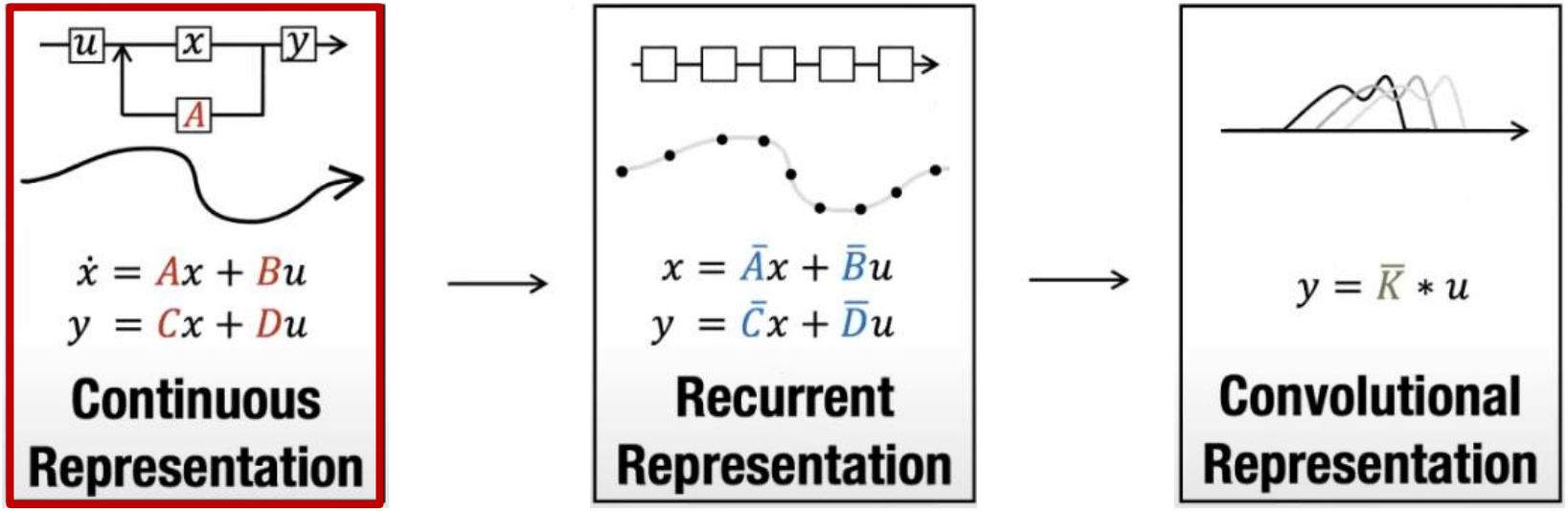
Background

- State Space Model
  - Overview



Background

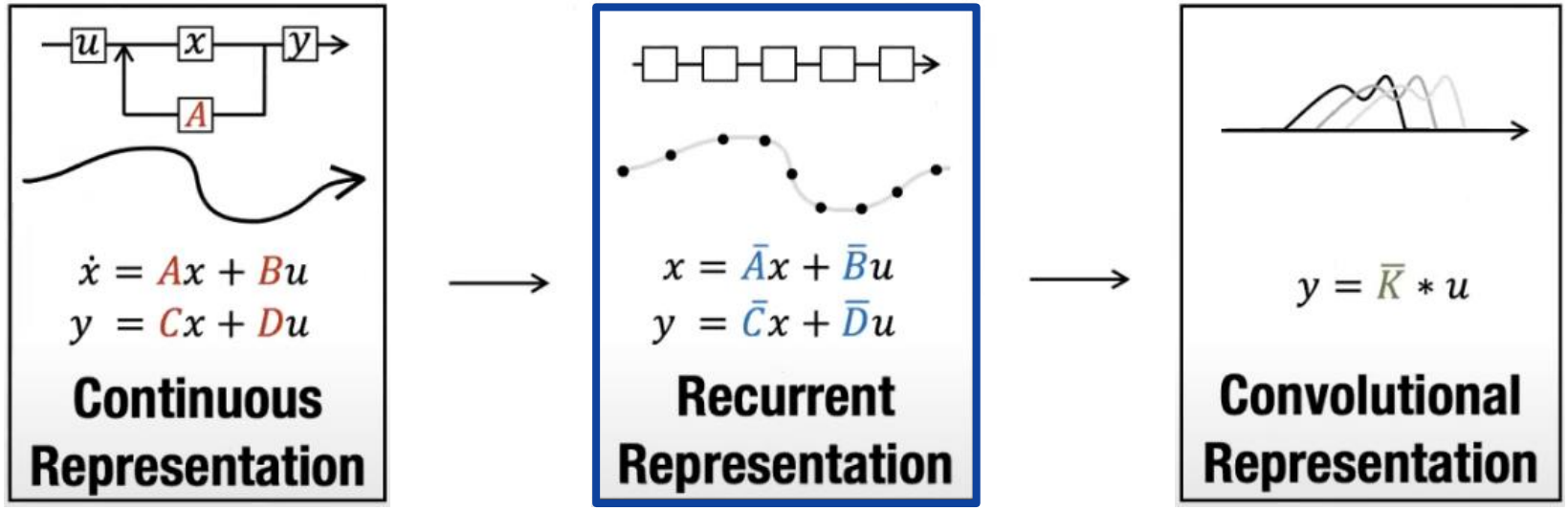
- State Space Model
  - Overview



Operate on **Signals** and Sequences

Background

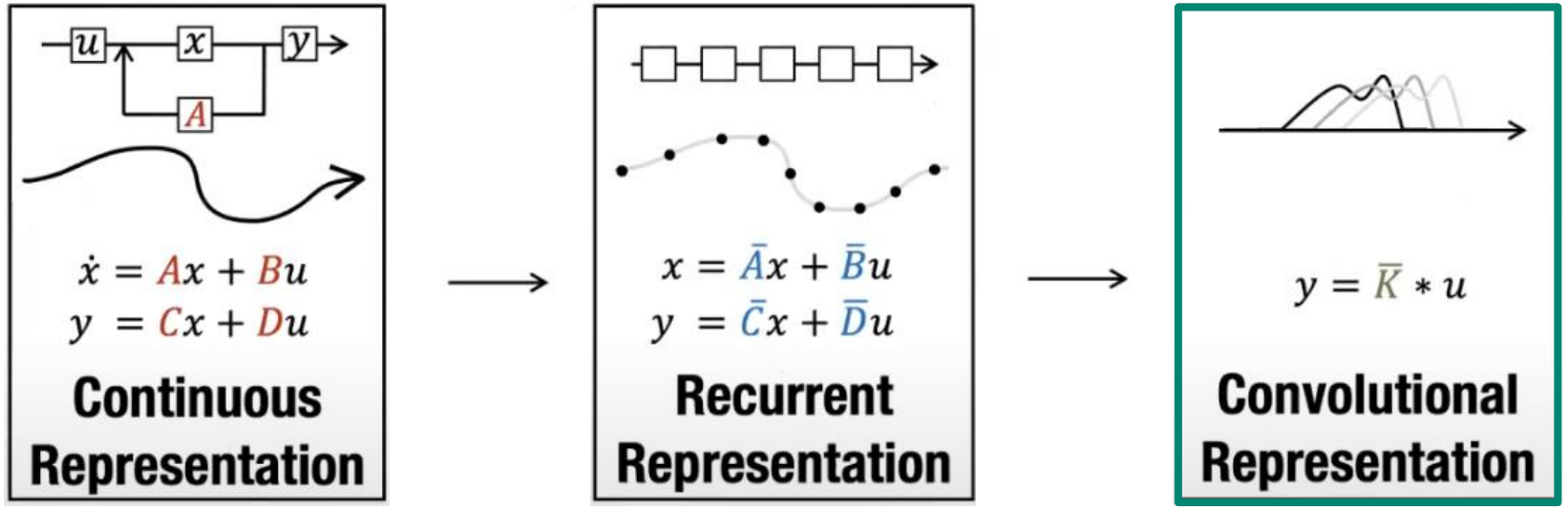
- State Space Model
  - Overview



Efficient **Online** / Autoregressive Computaiton

Background

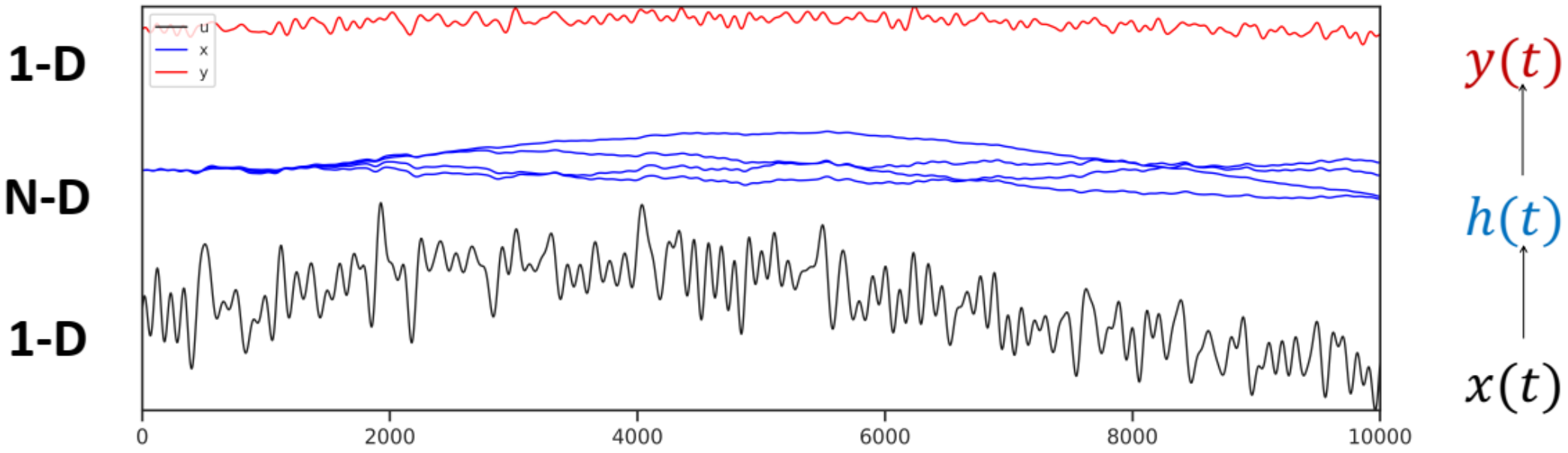
- State Space Model
  - Overview



Efficient **Parallelizable** Computation

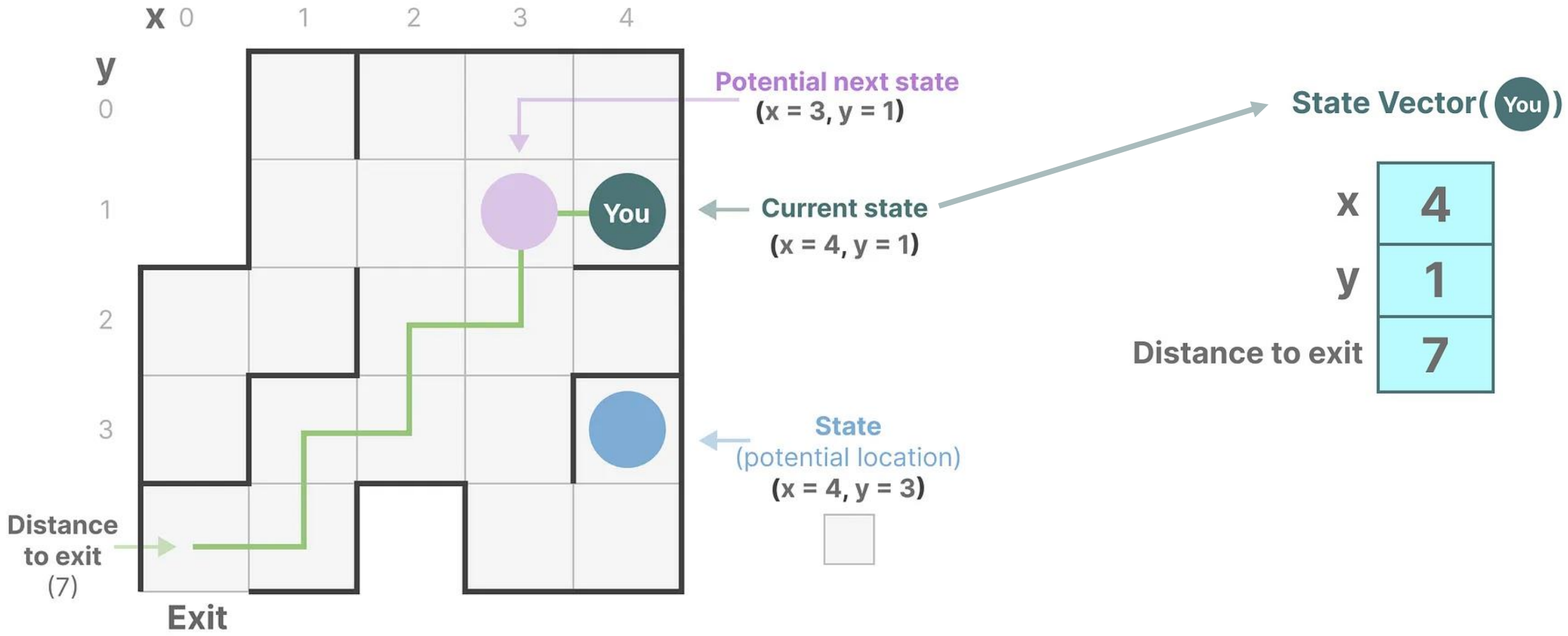
Background

- State Space Model
  - Used in control engineering and system identification.
  - Contains the minimum number of variables that fully describe a system.
  - ➢ Mathematically represent a problem by defining a system's possible states.



Background

- State Space Model



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - Variable
    - $x(t)$: input function or sequence
    - $h(t)$: hidden state
    - $y(t)$: output
    - t: time

Background

- State Space Model

State equation: $h'(t) = Ah(t) + Bx(t)$

Output equation: $y(t) = Ch(t) + Dx(t)$

$x(t) \in \mathbb{R}^1$
$h(t) \in \mathbb{R}^n$
$y(t) \in \mathbb{R}^1$

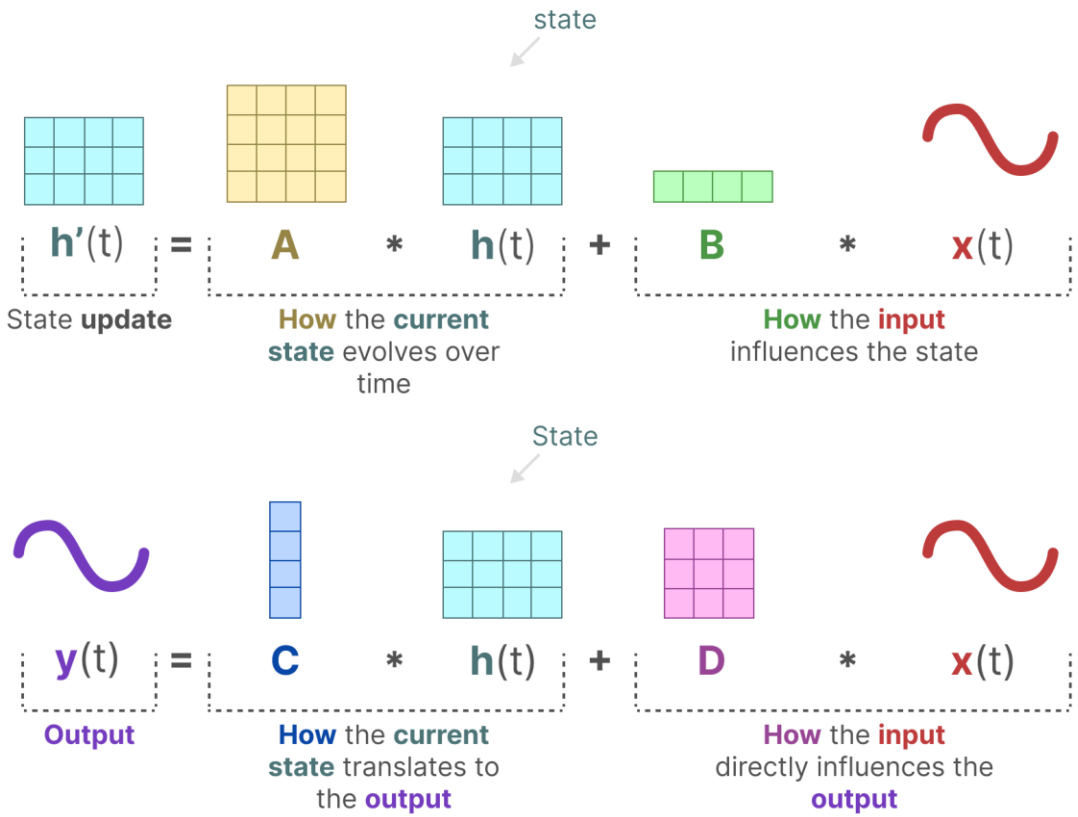
$A \in \mathbb{R}^{n \times n}$
$B \in \mathbb{R}^{n \times 1}$
$C \in \mathbb{R}^{1 \times n}$
$D \in \mathbb{R}^{1 \times 1}$

$Ah(t) \in \mathbb{R}^n$
$Bx(t) \in \mathbb{R}^n$
$Ch(t) \in \mathbb{R}^1$
$Dx(t) \in \mathbb{R}^1$

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model

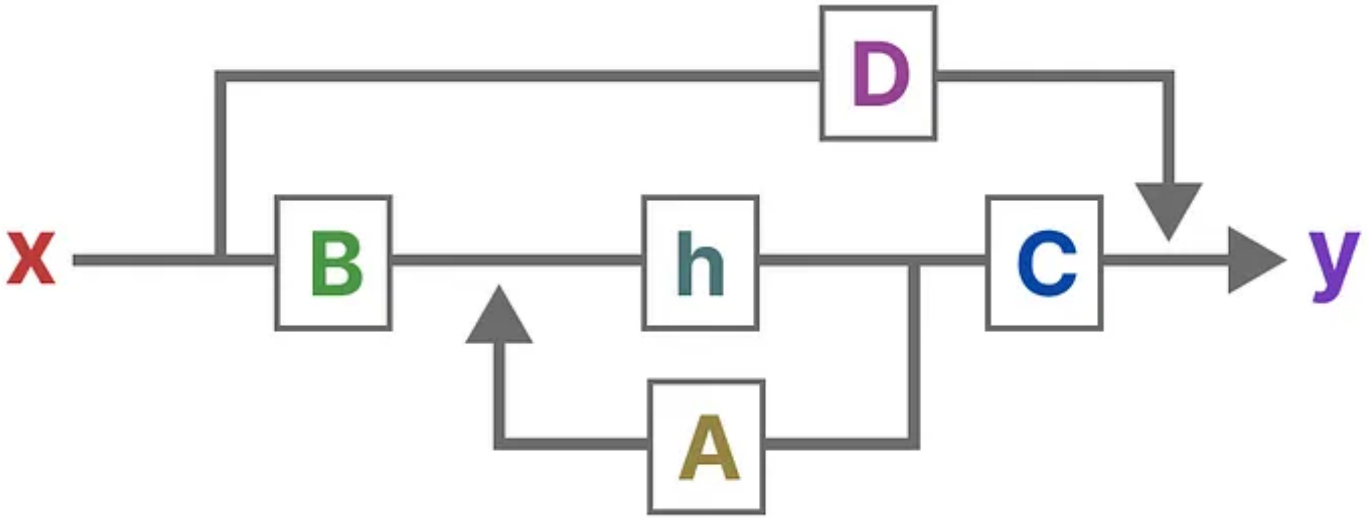


$$x(t) \in \mathbb{R}^1$$
$$h(t) \in \mathbb{R}^n$$
$$y(t) \in \mathbb{R}^1$$

$$A \in \mathbb{R}^{n \times n} \qquad Ah(t) \in \mathbb{R}^n$$
$$B \in \mathbb{R}^{n \times 1} \qquad Bx(t) \in \mathbb{R}^n$$
$$C \in \mathbb{R}^{1 \times n} \qquad Ch(t) \in \mathbb{R}^1$$
$$D \in \mathbb{R}^{1 \times 1} \qquad Dx(t) \in \mathbb{R}^1$$

Background

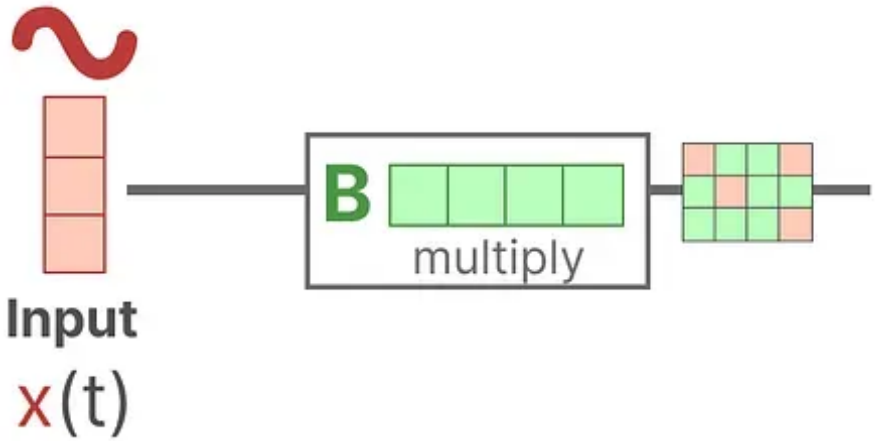
- State Space Model



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

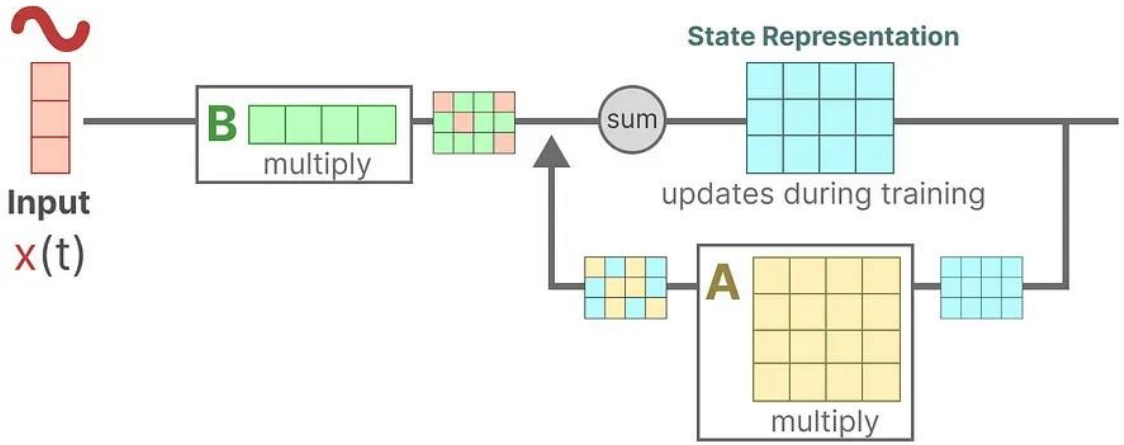
- State Space Model



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

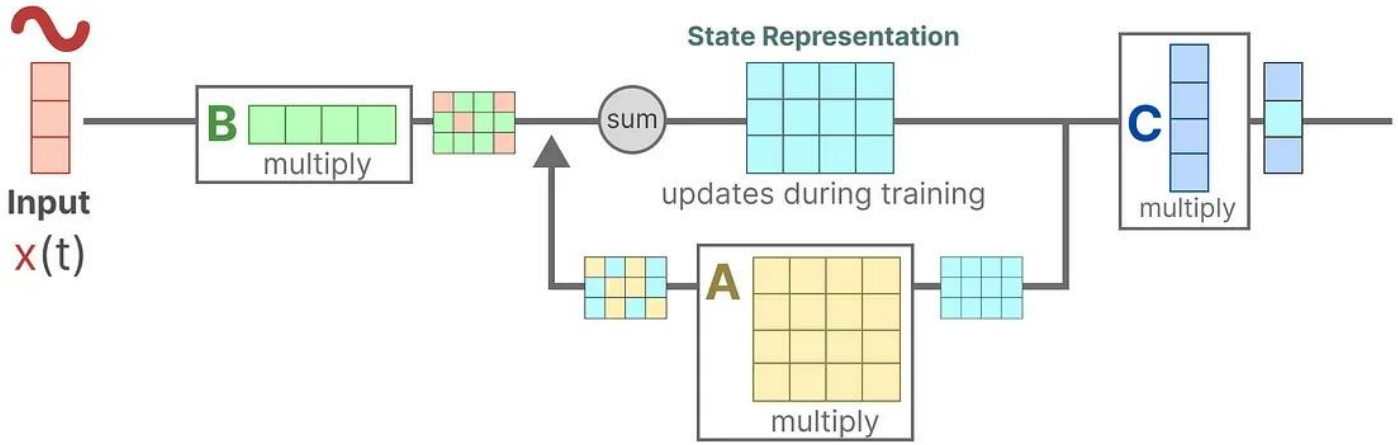
Background

- State Space Model



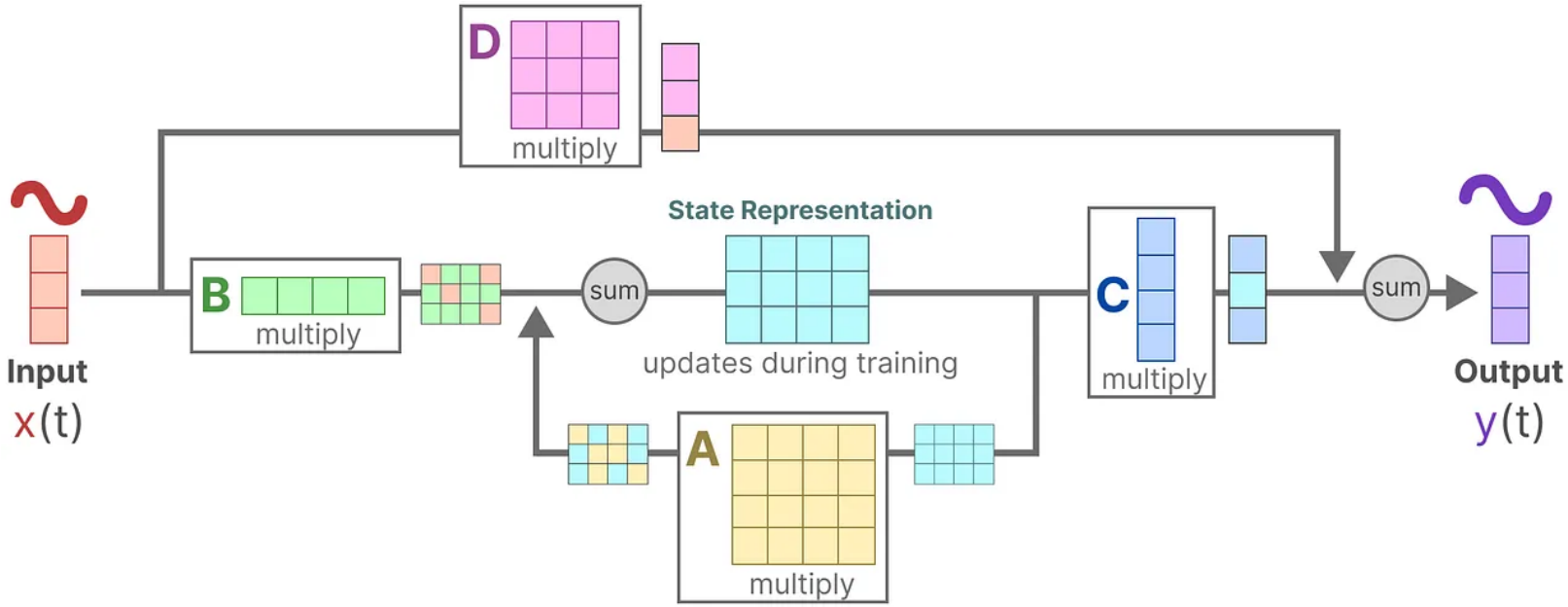
Background

- State Space Model



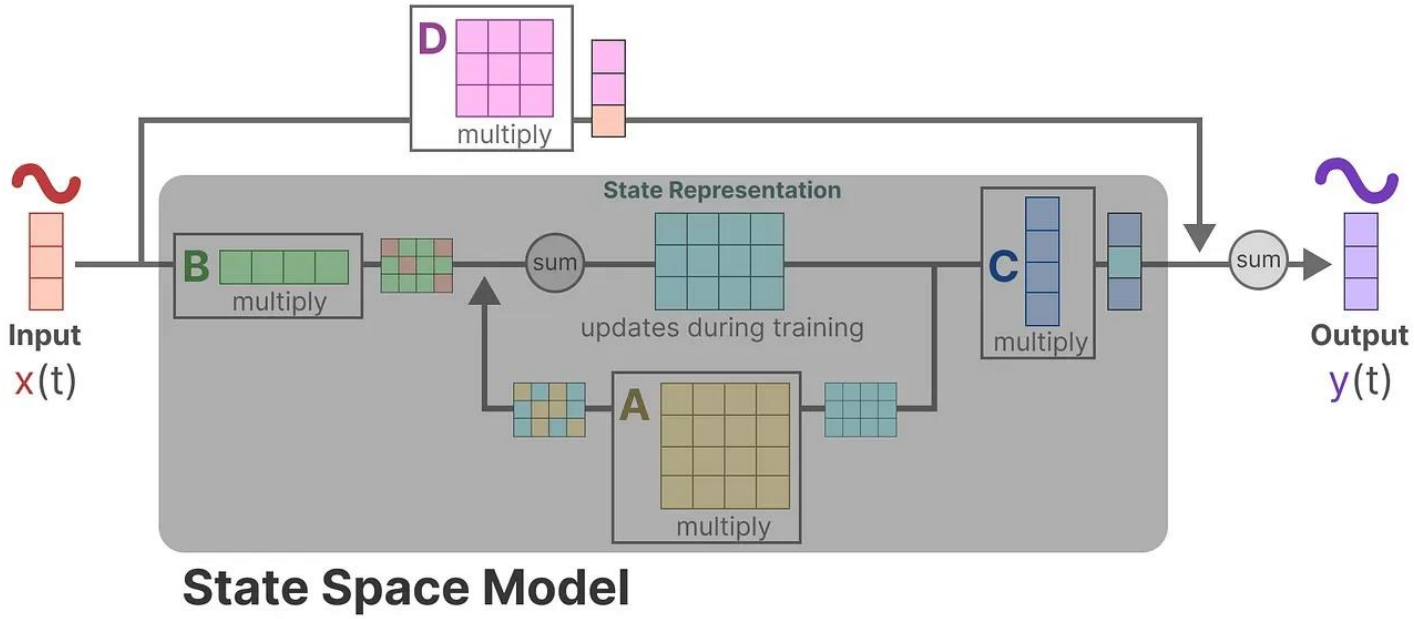
Background

- State Space Model



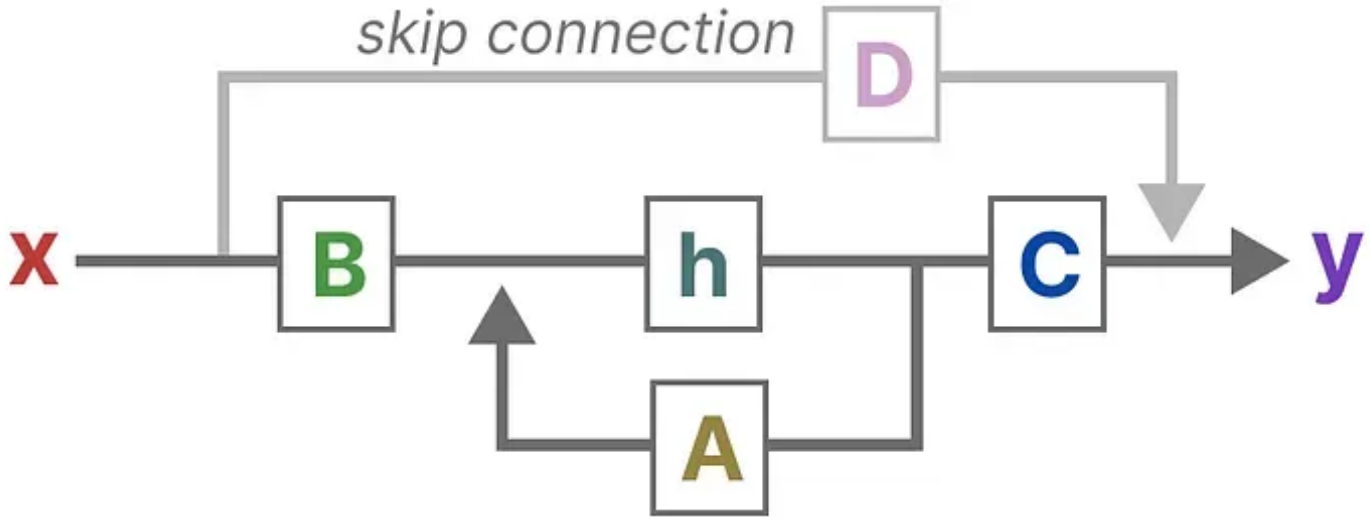
Background

- State Space Model



Background

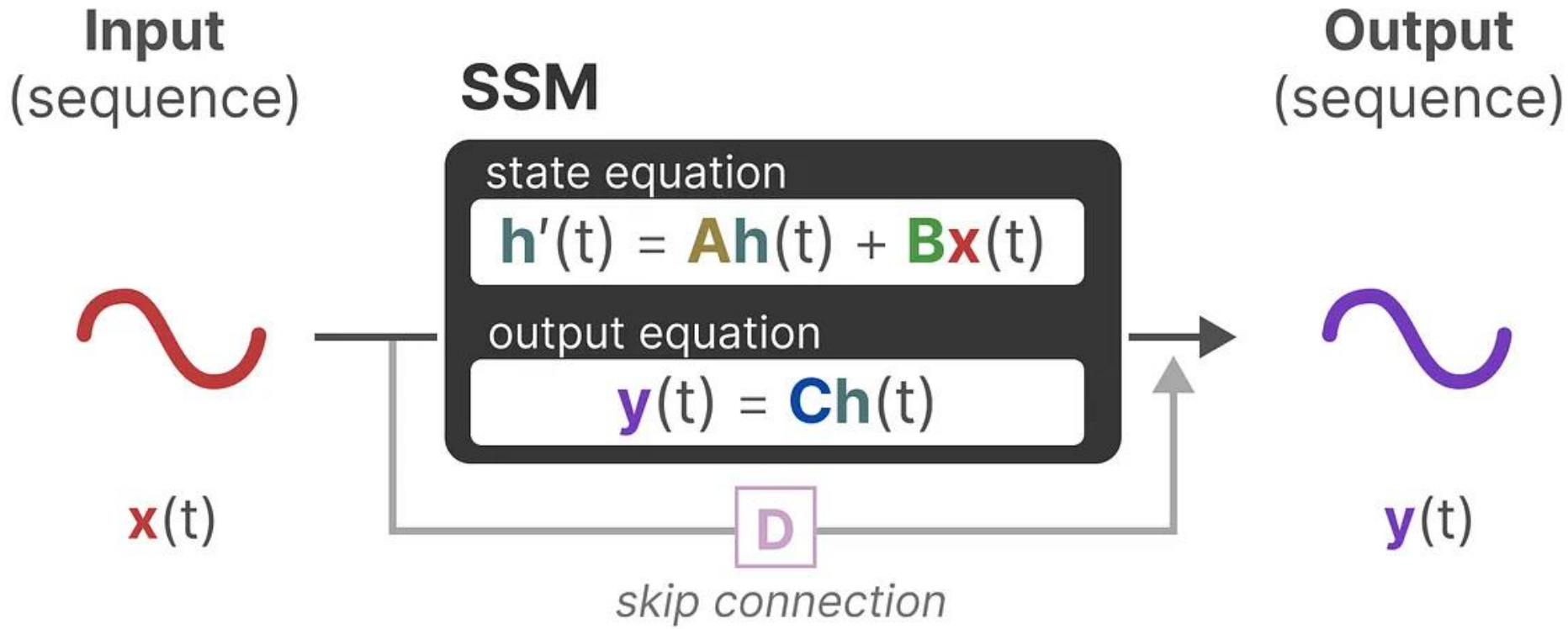
- State Space Model



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - Linear Time Invariant(LTI) System



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - Ordinary Differential Equation: Euler's method

$$\frac{d}{dt}h(t) = h'(t) = \mathrm{A}h(t) + \mathrm{B}x(t)$$
$$y(t) = \mathrm{C}h(t)$$

Discretization!

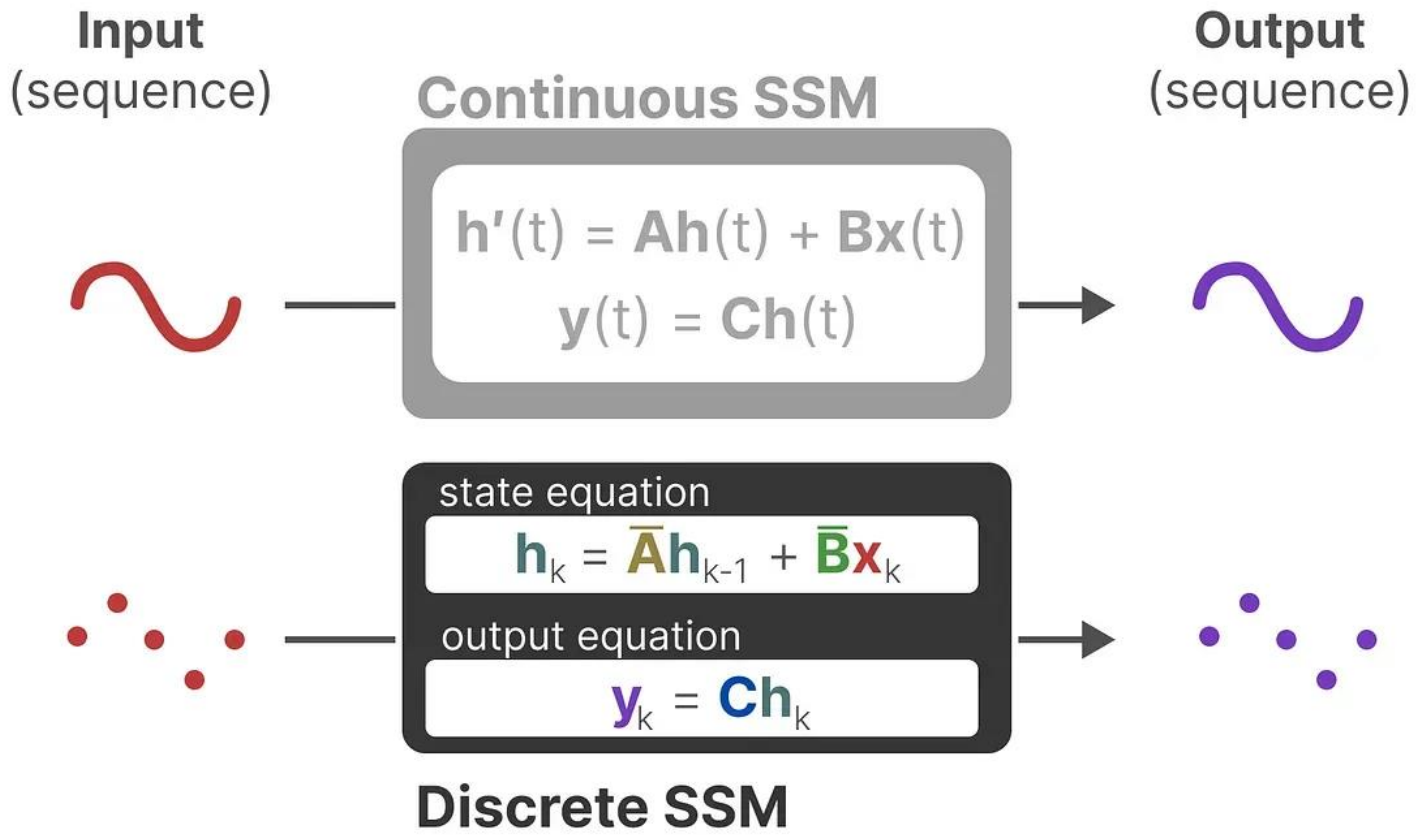
$$h_t = \overline{\mathrm{A}}h_{t-1} + \overline{\mathrm{B}}x_t$$
$$y_t = \mathrm{C}h_t$$

$$h(t+\Delta) \cong \Delta\big(\mathrm{A}h(t) + \mathrm{B}x(t)\big) + h(t)$$
$$= \Delta\mathrm{A}h(t) + \Delta\mathrm{B}x(t) + h(t)$$
$$= (\mathrm{I} + \Delta\mathrm{A})h(t) + \Delta\mathrm{B}x(t)$$
$$= \overline{\mathrm{A}}h(t) + \overline{\mathrm{B}}x(t)$$

$$\frac{d}{dt}h(t) = \lim_{\Delta\to 0}\frac{h(t+\Delta) - h(t)}{\Delta}$$
$$\cong \frac{h(t+\Delta) - h(t)}{\Delta}$$

Background

- State Space Model
  - Linear Time Invariant(LTI) System



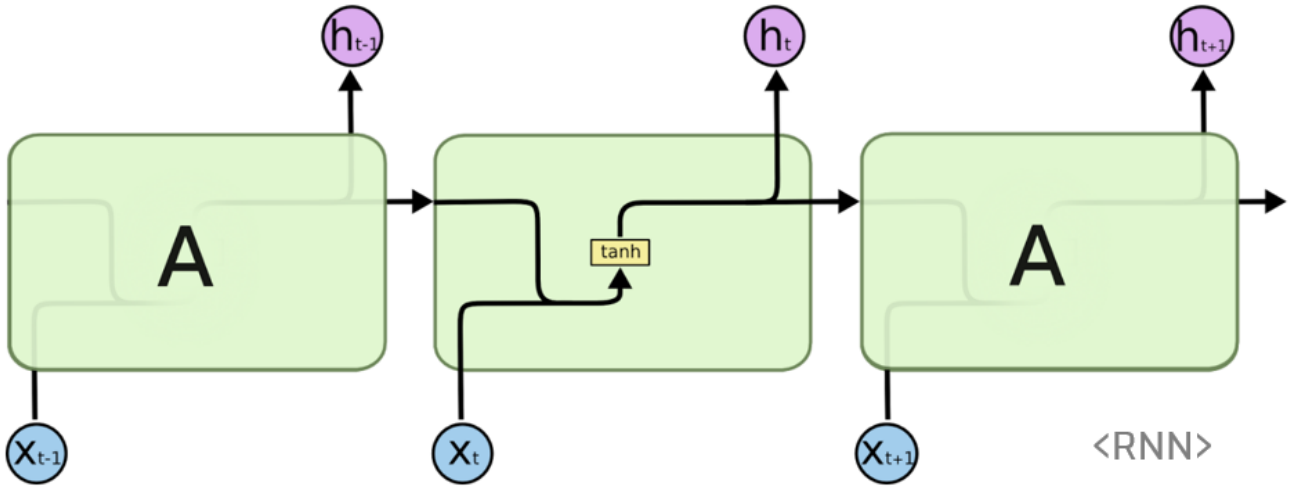
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Recurrent Representation

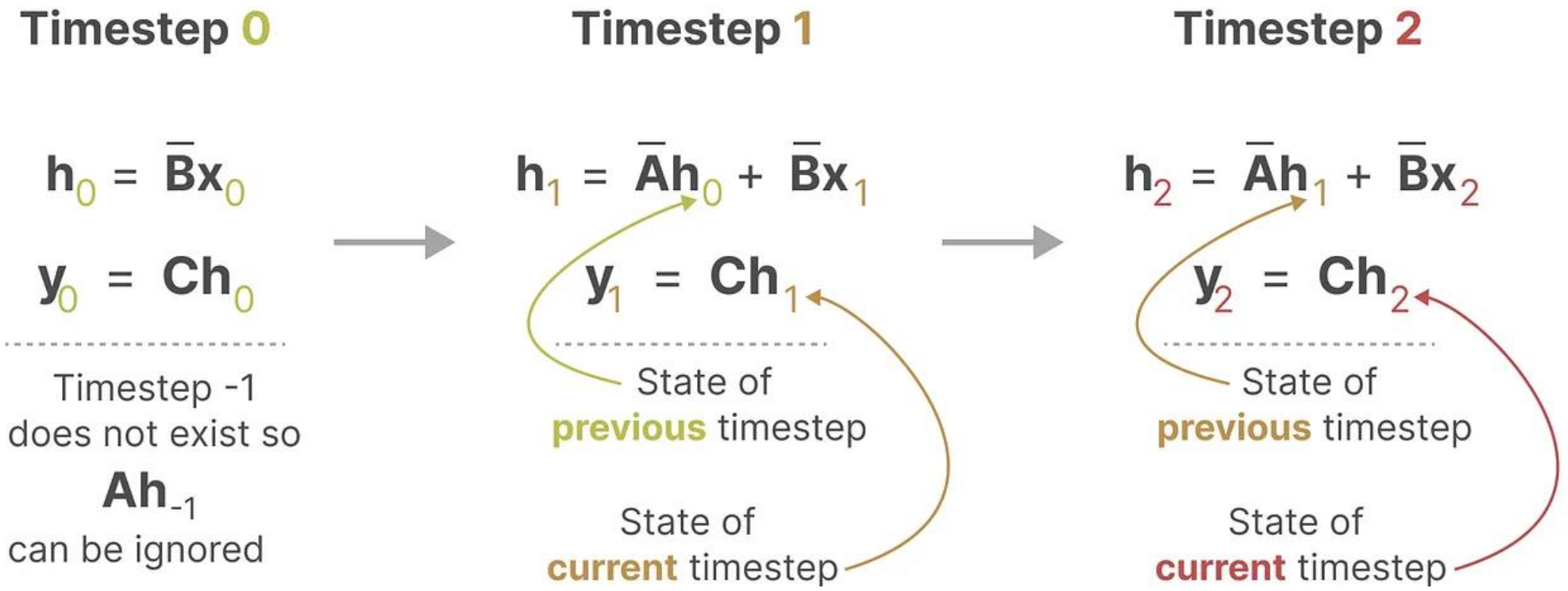
$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

=



Background

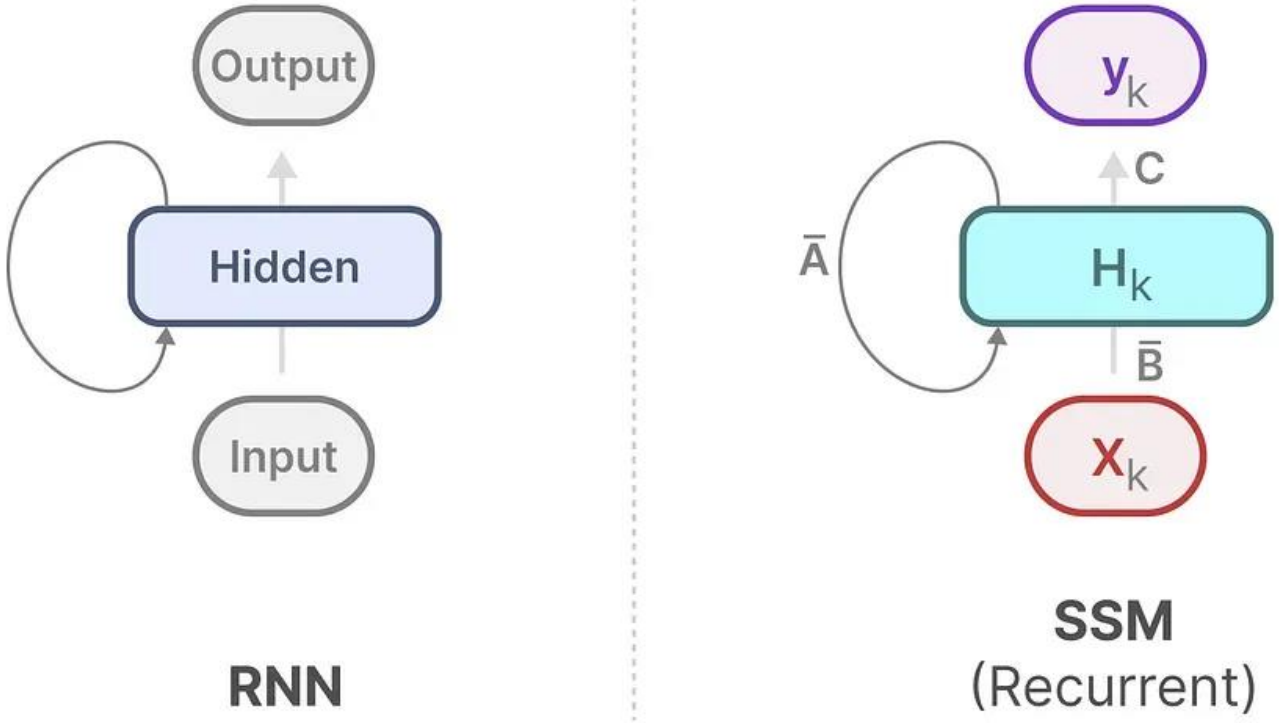
- State Space Model
  - The Recurrent Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

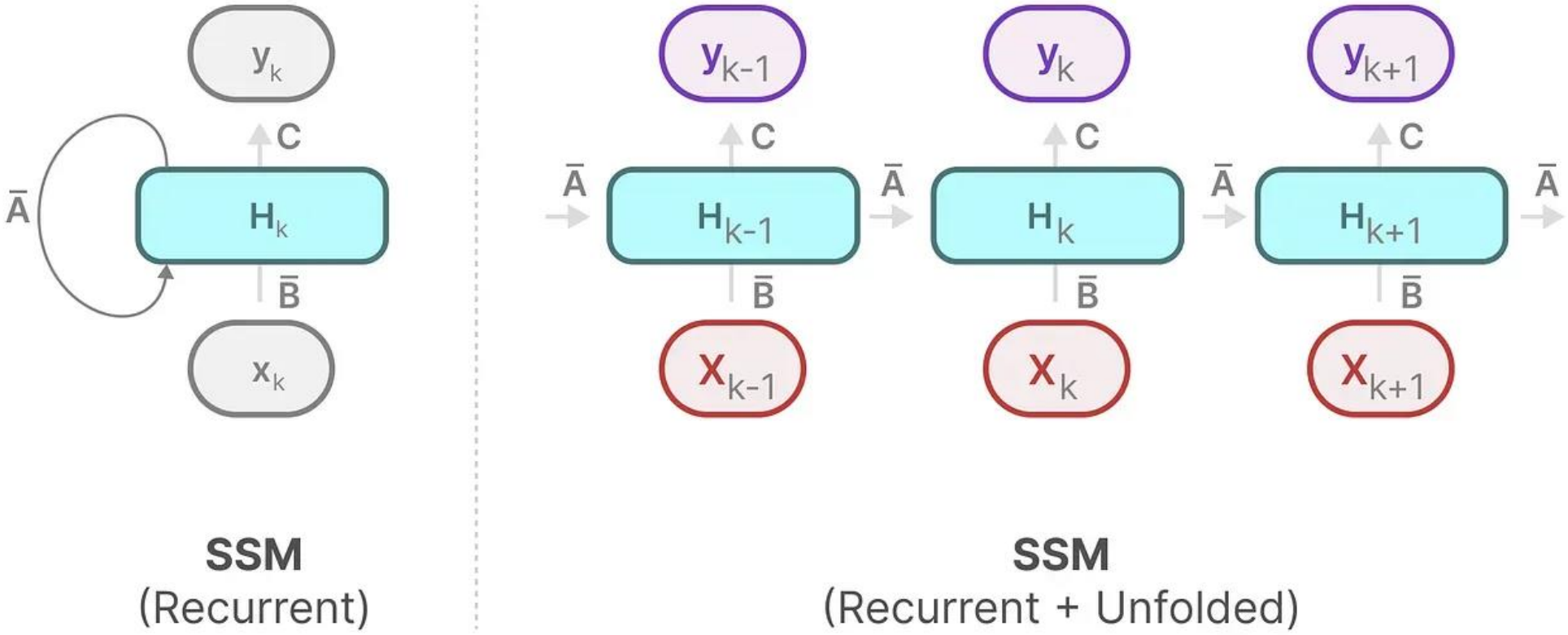
- State Space Model
  - The Recurrent Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Recurrent Representation

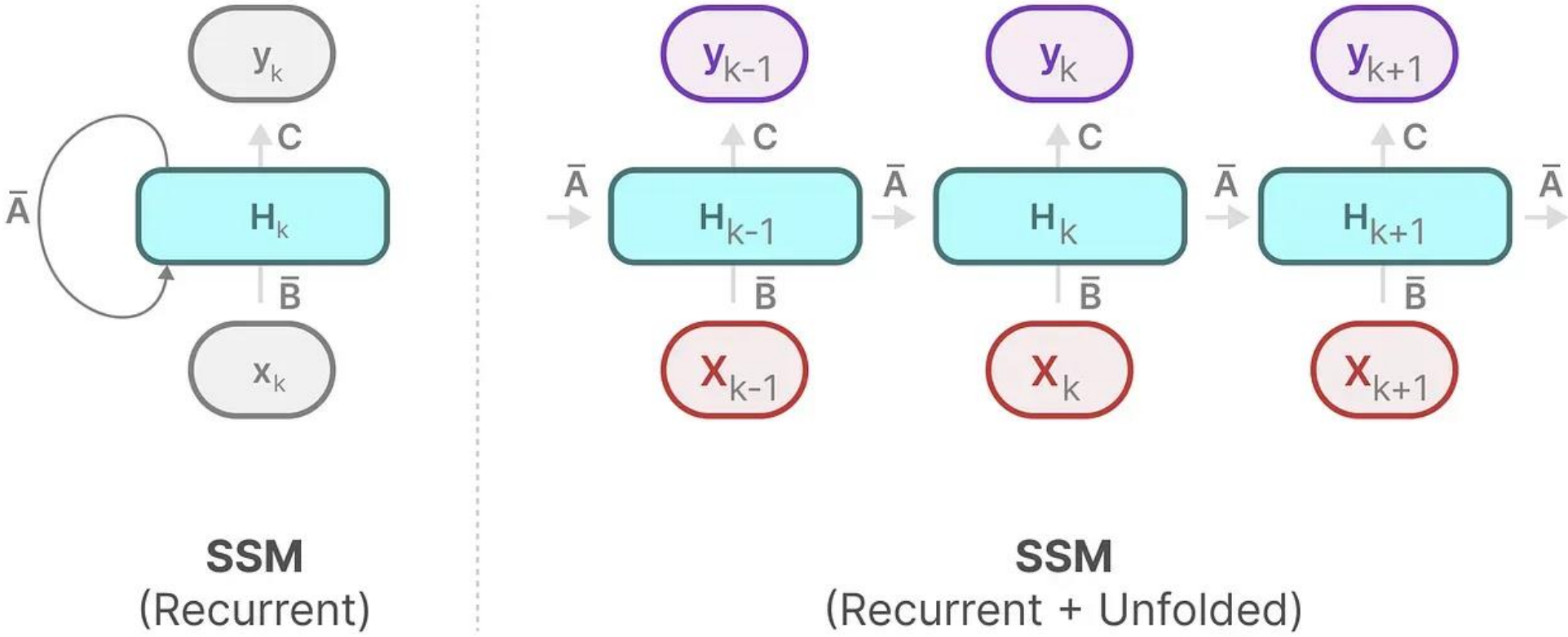


https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Recurrent Representation

So what is the difference with RNN?



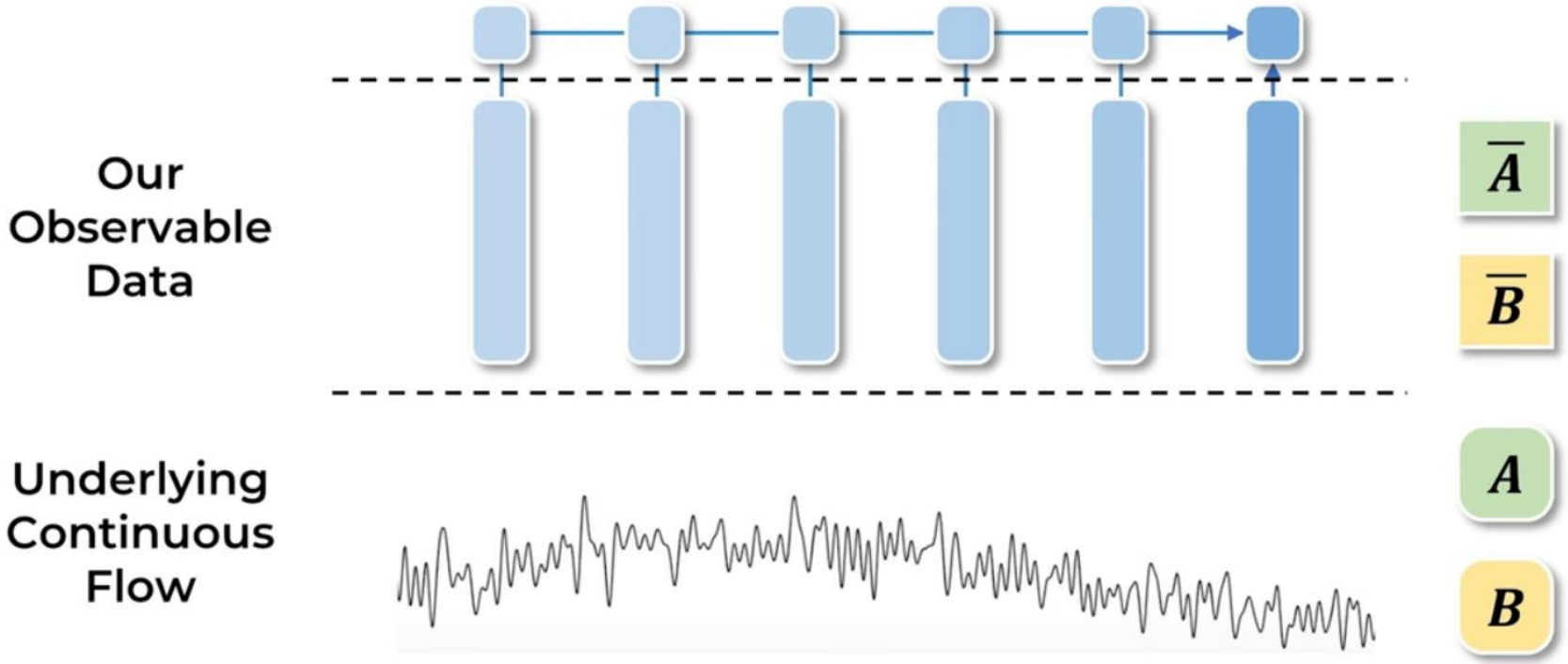
SSM
(Recurrent)

SSM
(Recurrent + Unfolded)

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

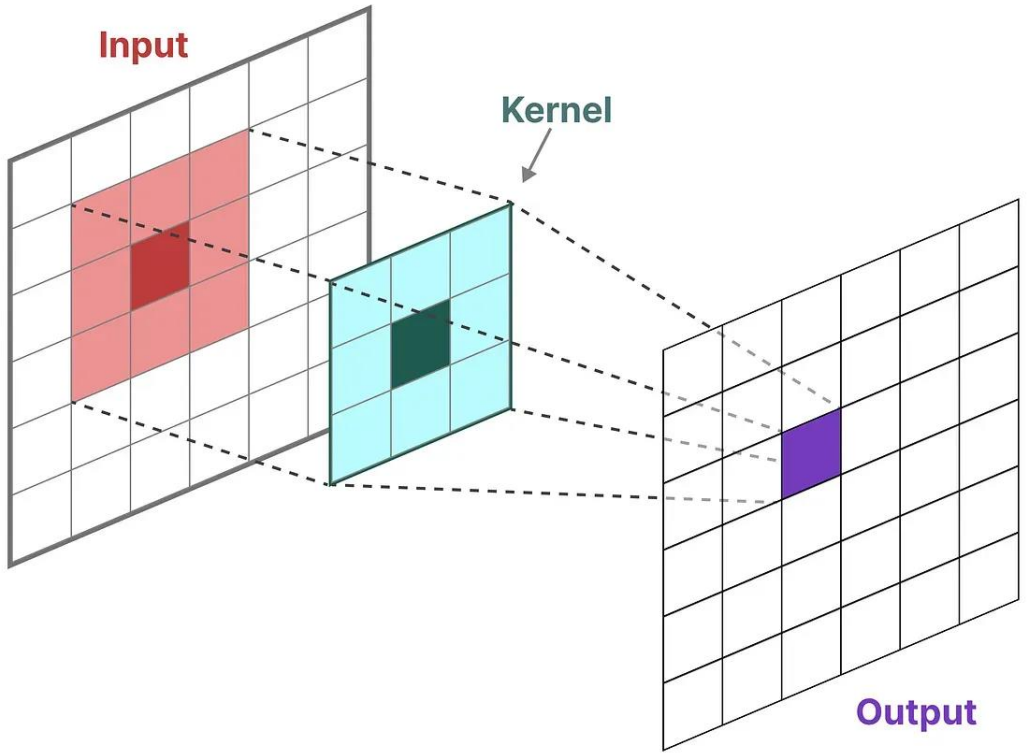
- State Space Model
  - The Recurrent Representation



https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

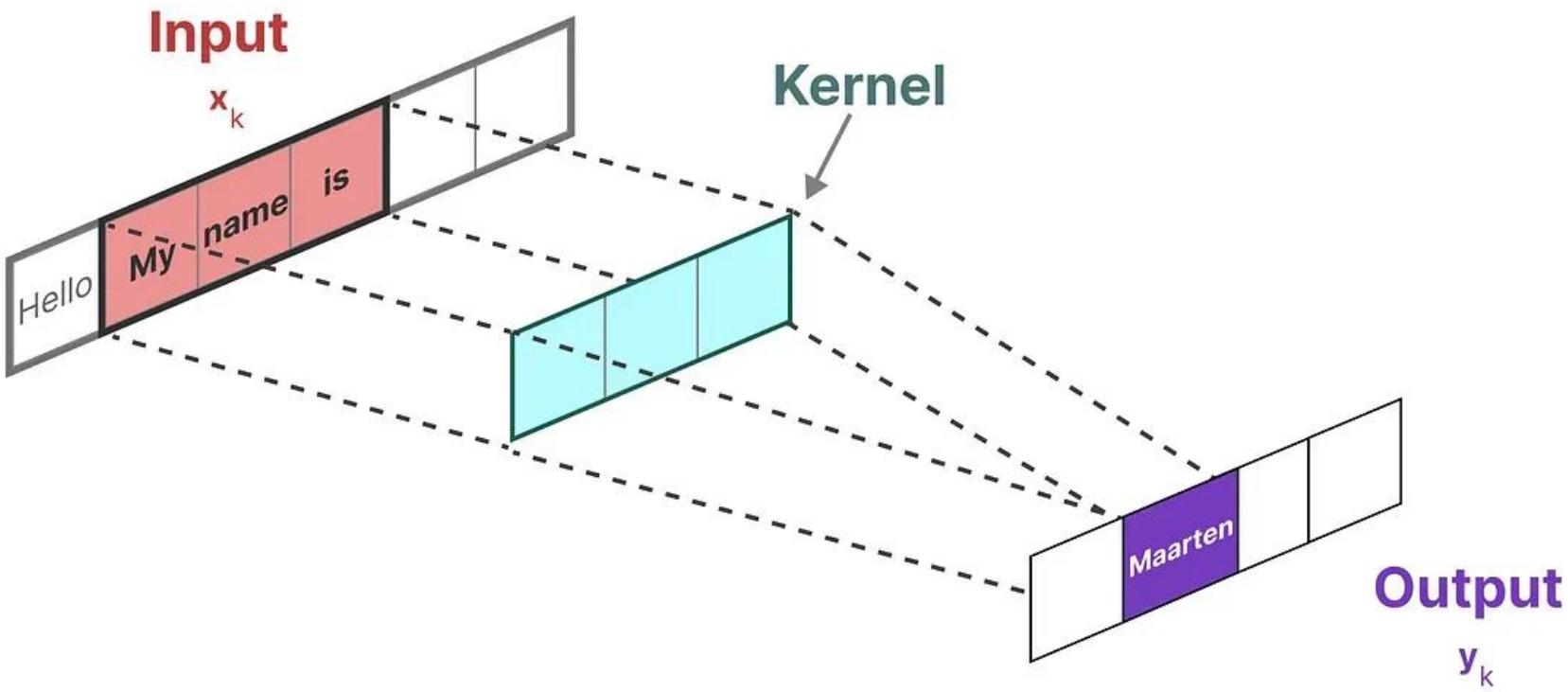
- State Space Model
  - The Convolutional Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

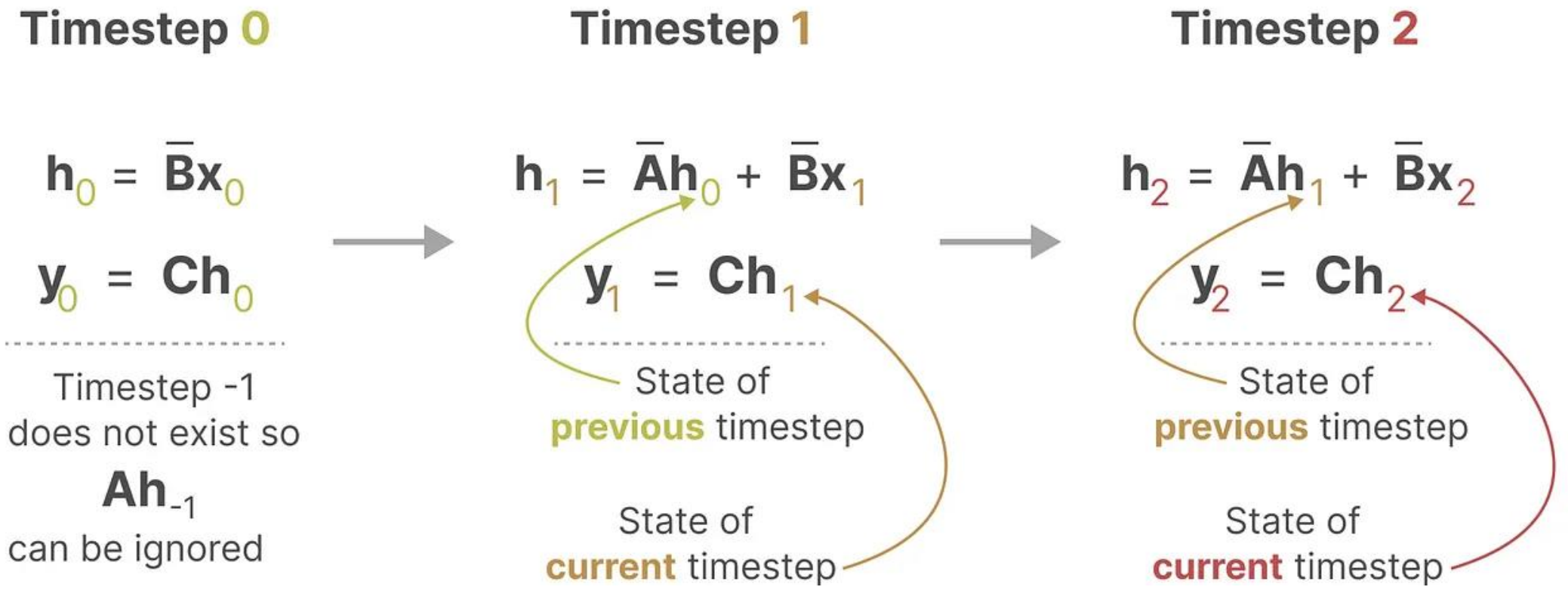
Background

- State Space Model
  - The Convolutional Representation



Background

- State Space Model
  - The Convolutional Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Convolutional Representation

$t = 0$

$h_0 = \overline{\mathrm{B}} x_0 \quad (\overline{\mathrm{A}} h_{-1} = 0)$

$y_0 = \mathrm{C} h_0$

$= \mathrm{C}\overline{\mathrm{B}} x_0$

$t = 1$

$h_1 = \overline{\mathrm{A}} h_0 + \overline{\mathrm{B}} x_1$

$= \overline{\mathrm{A}}\overline{\mathrm{B}} x_0 + \overline{\mathrm{B}} x_1$

$y_1 = \mathrm{C} h_1$

$= \mathrm{C}(\overline{\mathrm{A}}\overline{\mathrm{B}} x_0 + \overline{\mathrm{B}} x_1)$

$t = 2$

$h_2 = \overline{\mathrm{A}} h_1 + \overline{\mathrm{B}} x_2$

$= \overline{\mathrm{A}}^2\overline{\mathrm{B}} x_0 + \overline{\mathrm{A}}\overline{\mathrm{B}} x_1 + \overline{\mathrm{B}} x_2$

$y_2 = \mathrm{C} h_2$

$= \mathrm{C}(\overline{\mathrm{A}}^2\overline{\mathrm{B}} x_0 + \overline{\mathrm{A}}\overline{\mathrm{B}} x_1 + \overline{\mathrm{B}} x_2)$

$t = 3$

$h_3 = \overline{\mathrm{A}} h_2 + \overline{\mathrm{B}} x_3$

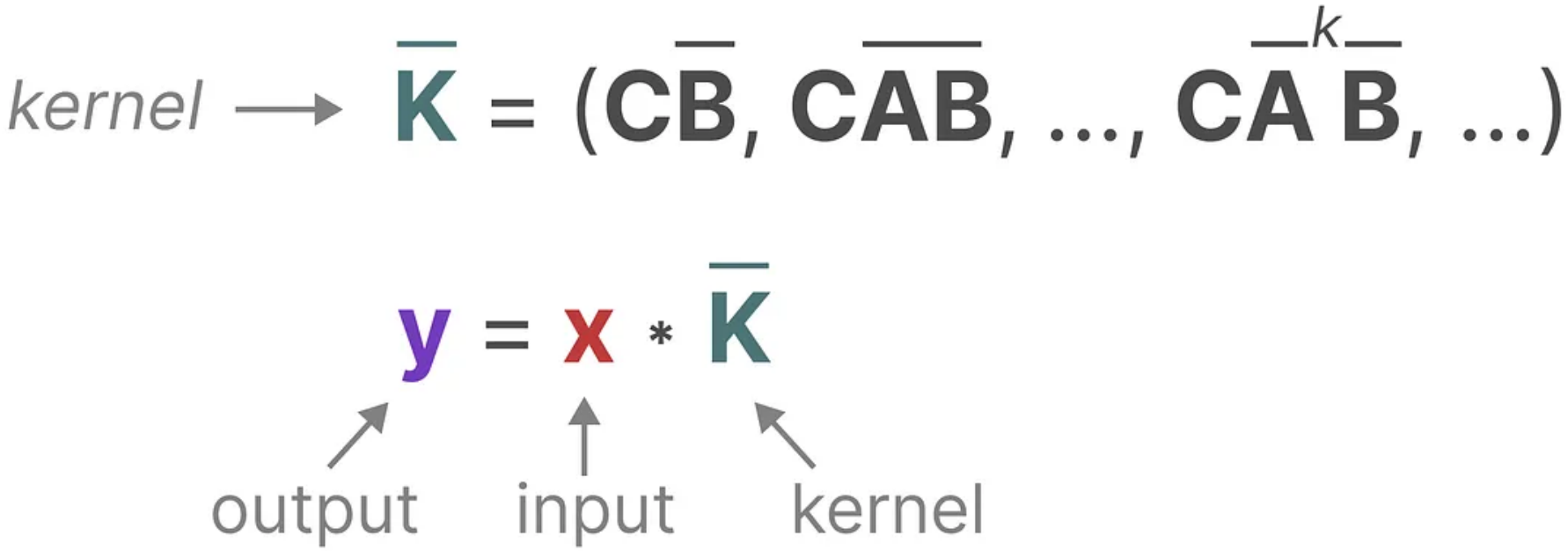
$= \overline{\mathrm{A}}^3\overline{\mathrm{B}} x_1 + \overline{\mathrm{A}}^2\overline{\mathrm{B}} x_2 + \overline{\mathrm{A}}\overline{\mathrm{B}} x_3$

$y_3 = \mathrm{C} h_3$

$= \mathrm{C}\big(\overline{\mathrm{A}}^3\overline{\mathrm{B}} x_1 + \overline{\mathrm{A}^2}\overline{\mathrm{B}} x_2 + \overline{\mathrm{A}}\overline{\mathrm{B}} x_1\big)$

Background

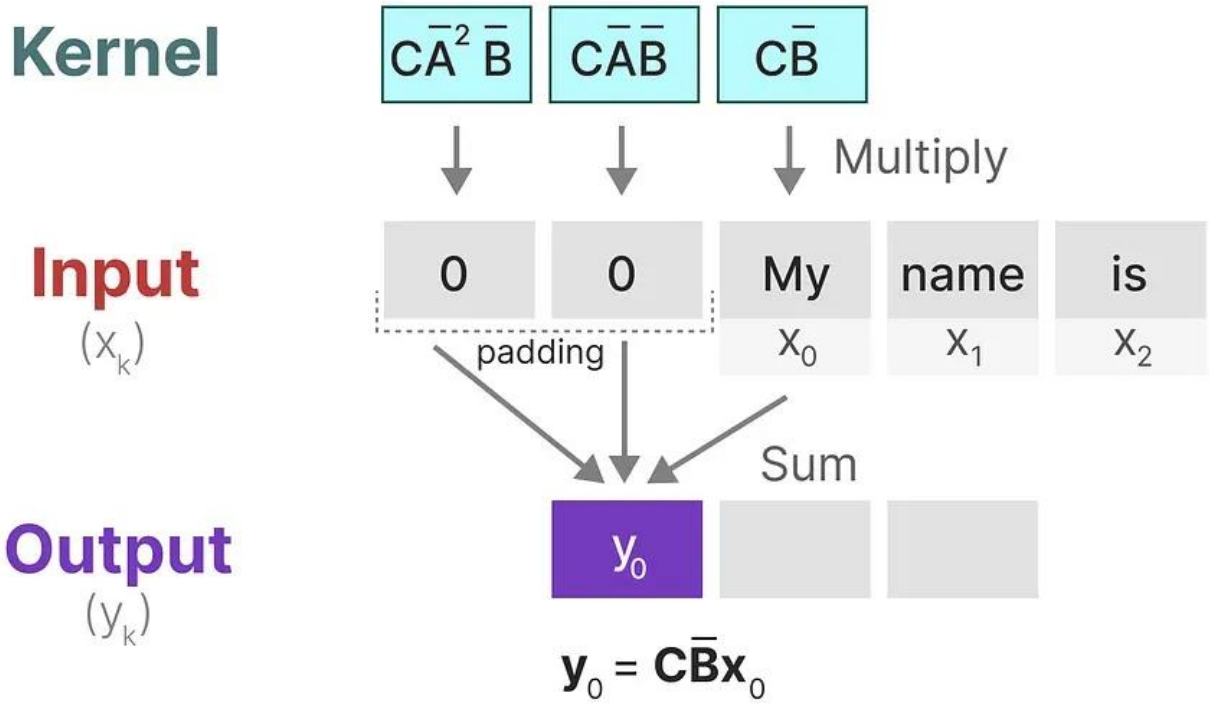
- State Space Model
  - The Convolutional Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

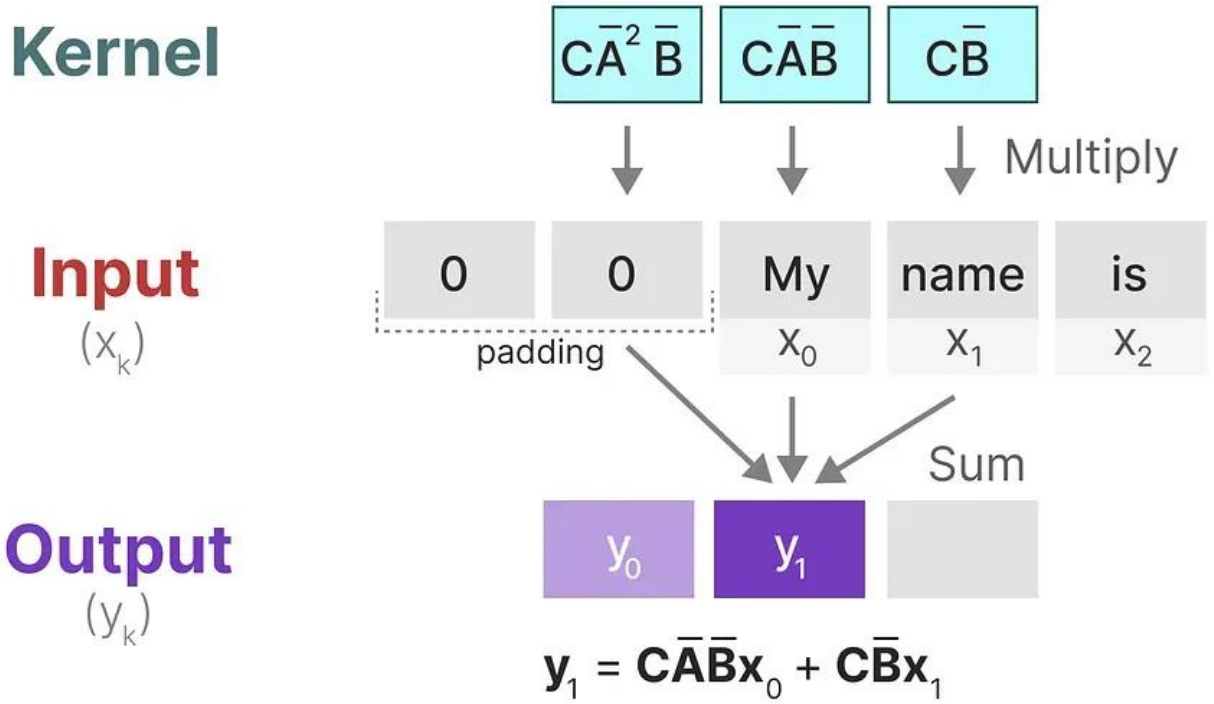
- State Space Model
  - The Convolutional Representation



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Convolutional Representation

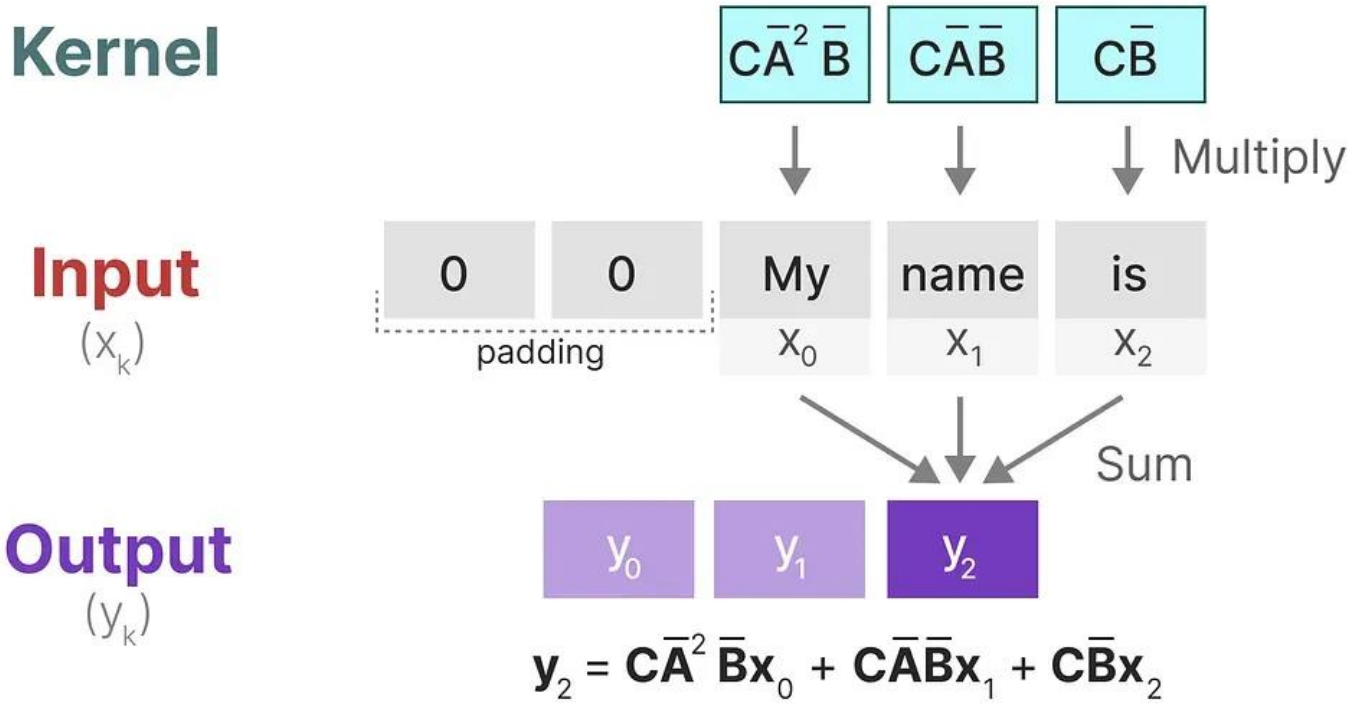


https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - The Convolutional Representation

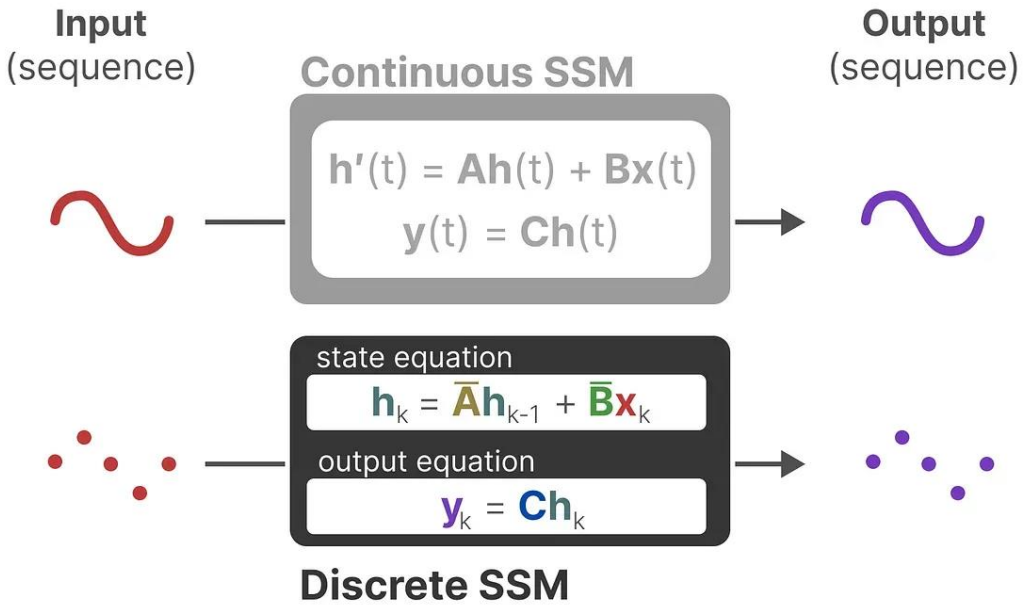
Can Parallelization!

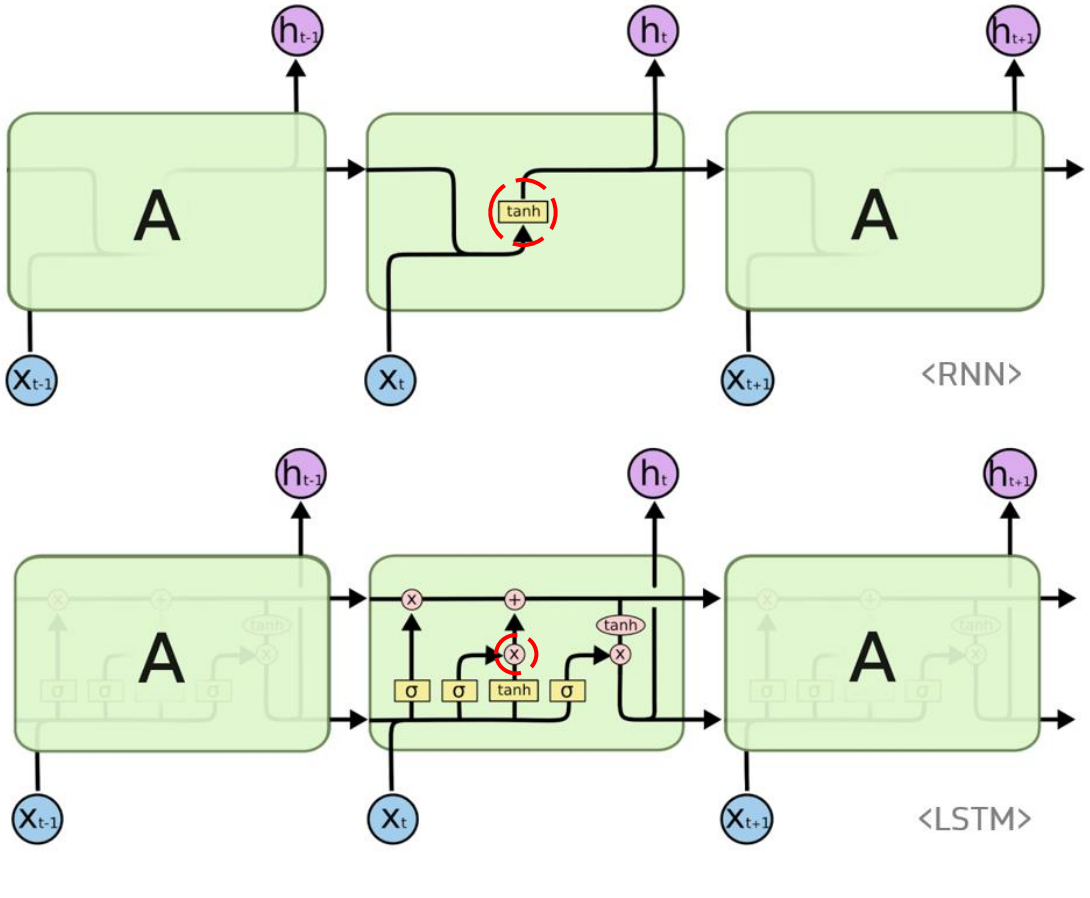


https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Background

- State Space Model
  - Why not RNN?
    1. Time-variant
    2. Non-linearlity

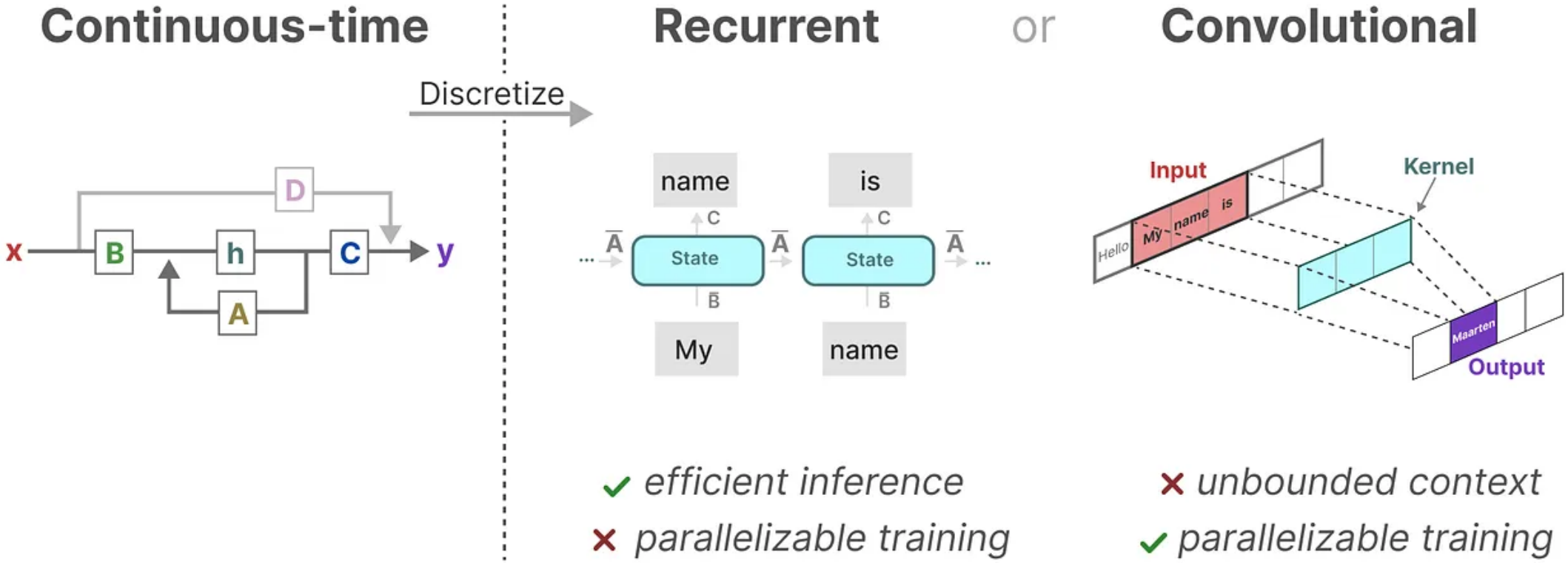




https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

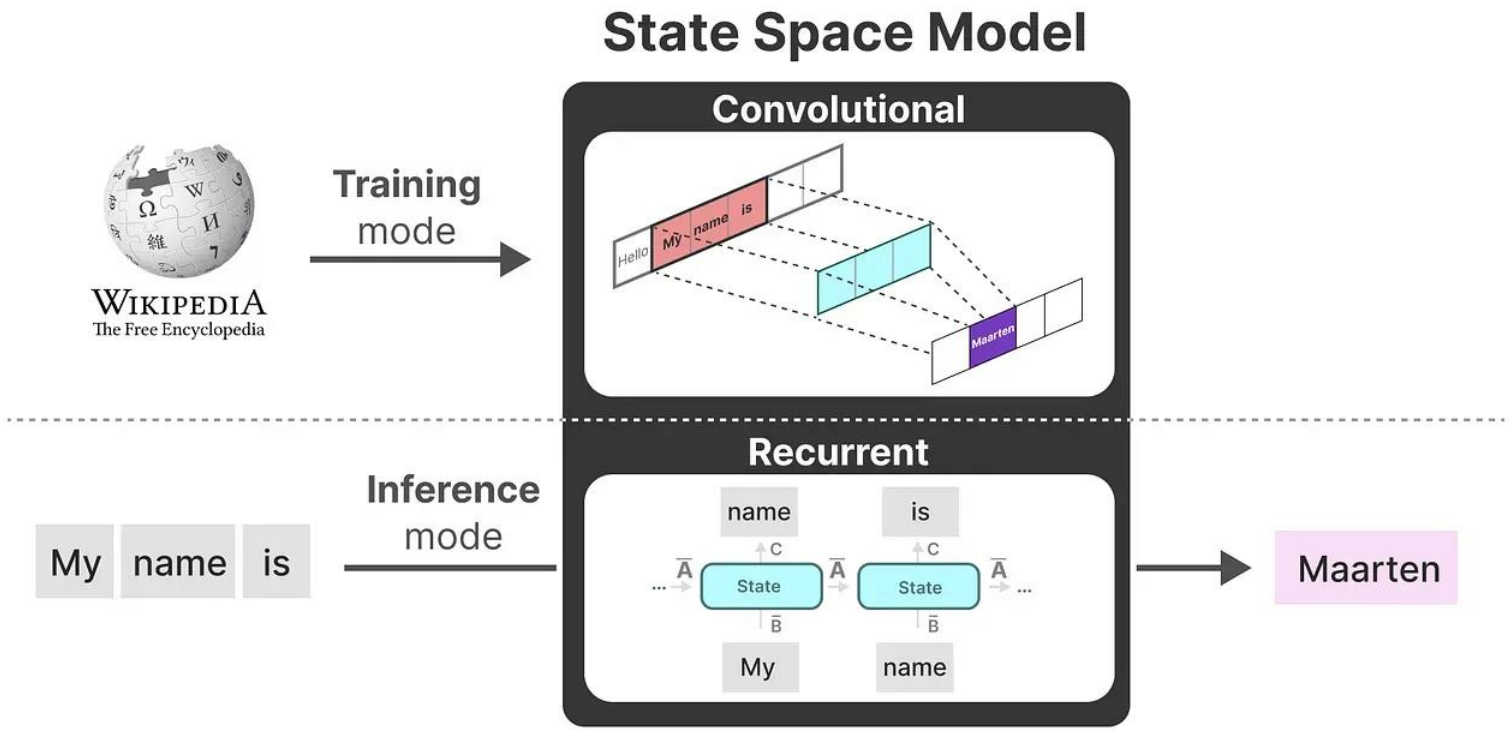
Background

- State Space Model
  - Training vs Inference



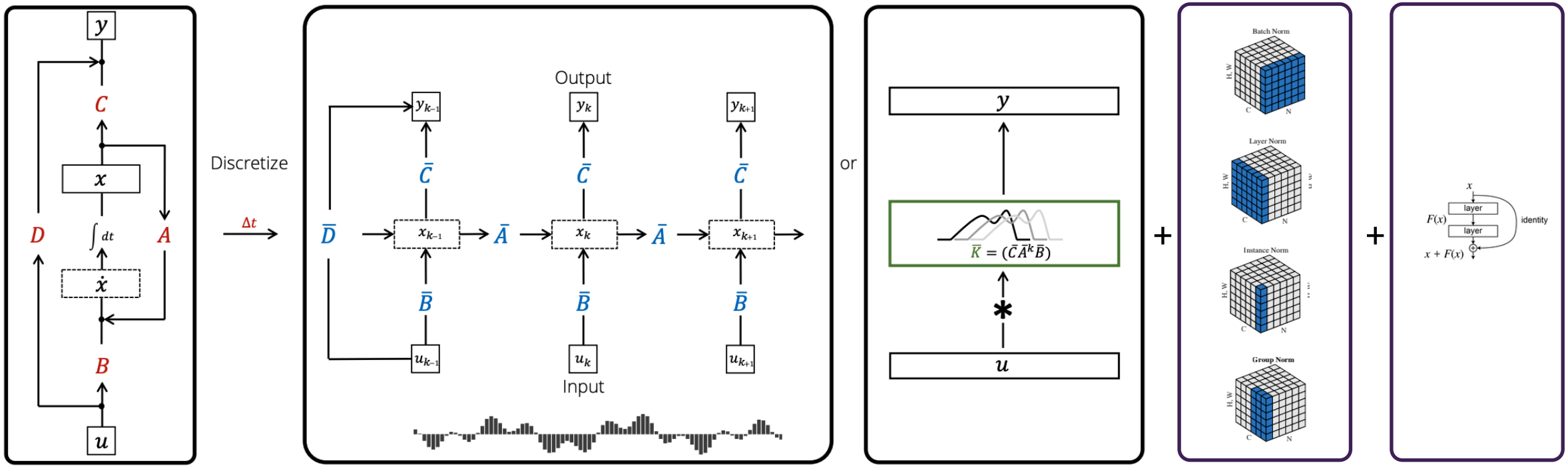
Background

- State Space Model
  - Training vs Inference



Background

- State Space Model
  - Linear State Space Layer(LSSL)



Continuous-time
- ✓ *continuous data*
- ✓ *irregular sampling*

Recurrent
- ✓ *unbounded context*
- ✓ *efficient inference*

Convolutional
- ✓ *local information*
- ✓ *parallelizable training*

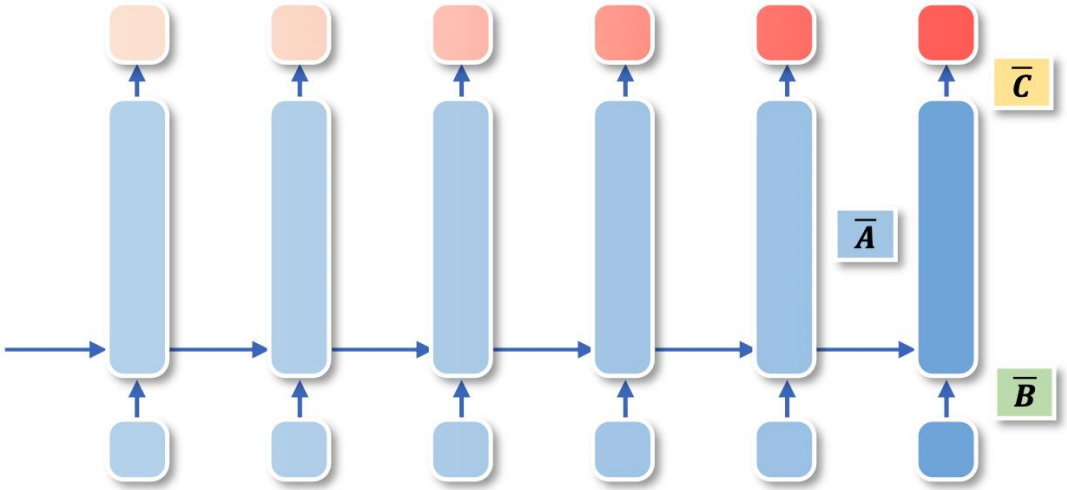
Normalization

Residual Connection

Combining Recurrent, Convolutional, and Continuous-time Models with Linear State-Space Layers
https://gaussian37.github.io/dl-concept-various_normalization/
https://en.wikipedia.org/wiki/Residual_neural_network

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

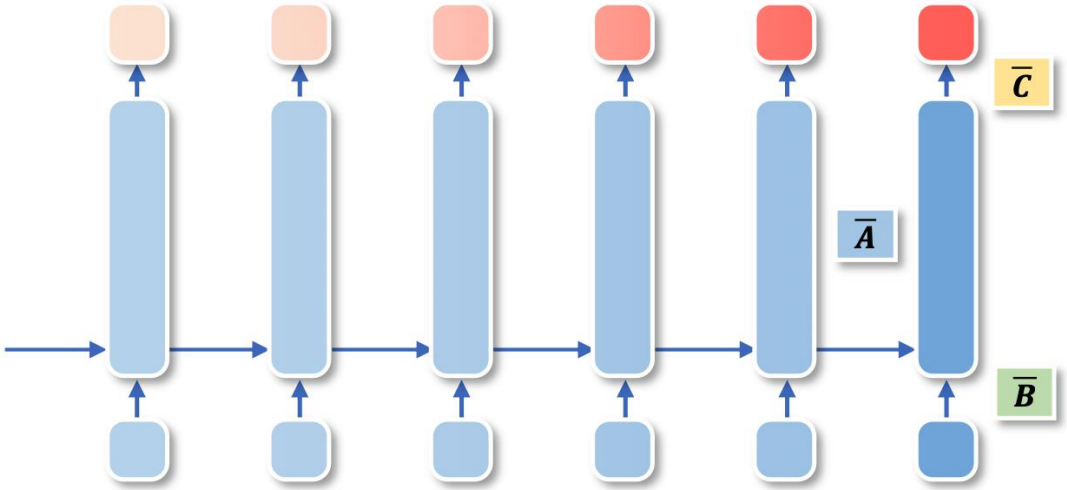
$$x_t \in \mathbb{R}^1$$
$$h_t \in \mathbb{R}^n$$
$$y_t \in \mathbb{R}^1$$

$$\overline{\mathrm{A}} \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}} \in \mathbb{R}^{n \times 1}$$
$$C \in \mathbb{R}^{1 \times n}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

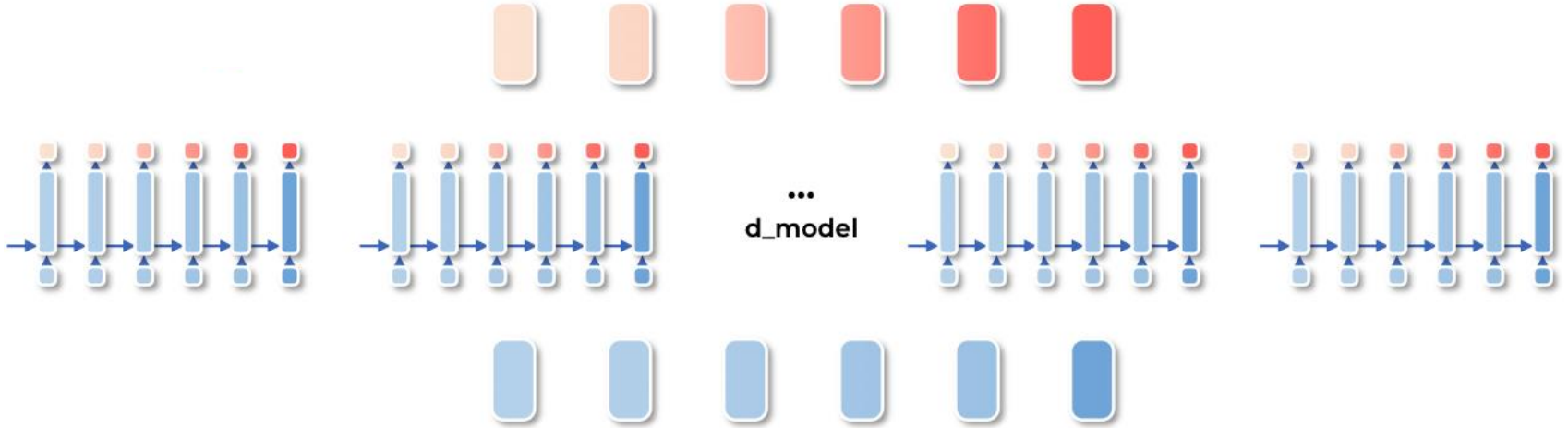
$$x_t \in \mathbb{R}^{d\_model}$$
$$h_t \in \mathbb{R}^{n}$$
$$y_t \in \mathbb{R}^{d\_model}$$

$$\overline{\mathrm{A}} \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}} \in \mathbb{R}^{n \times d\_model}$$
$$C \in \mathbb{R}^{d\_model \times n}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

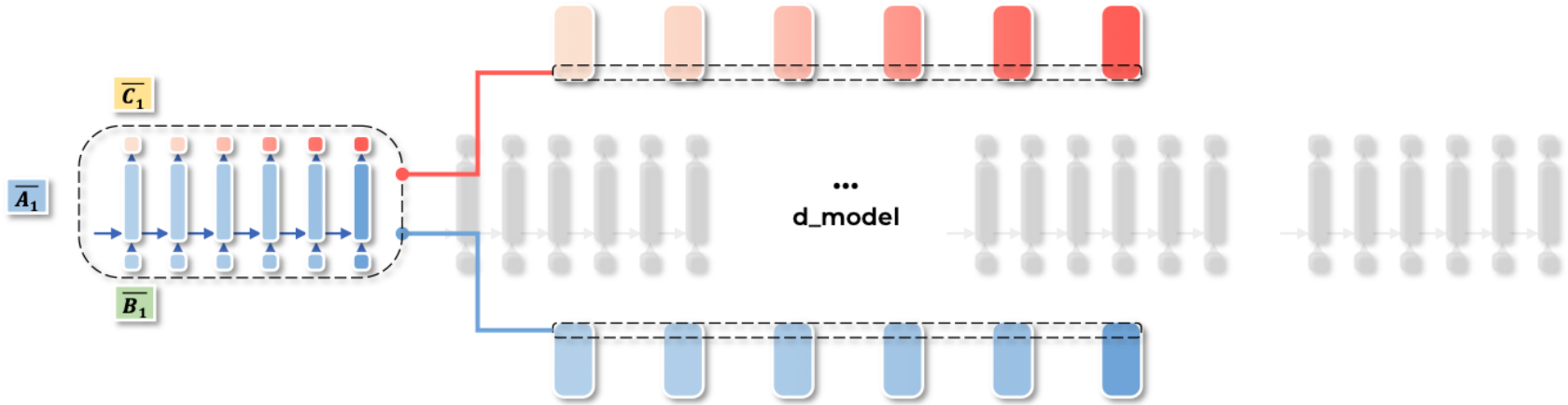
$$x_t \in \mathbb{R}^{d\_model}$$
$$h_t \in \mathbb{R}^{n}$$
$$y_t \in \mathbb{R}^{d\_model}$$

$$\overline{\mathrm{A}} \in \mathbb{R}^{n \times n \times d\_model}$$
$$\overline{\mathrm{B}} \in \mathbb{R}^{n \times 1 \times d\_model}$$
$$C \in \mathbb{R}^{1 \times n \times d\_model}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

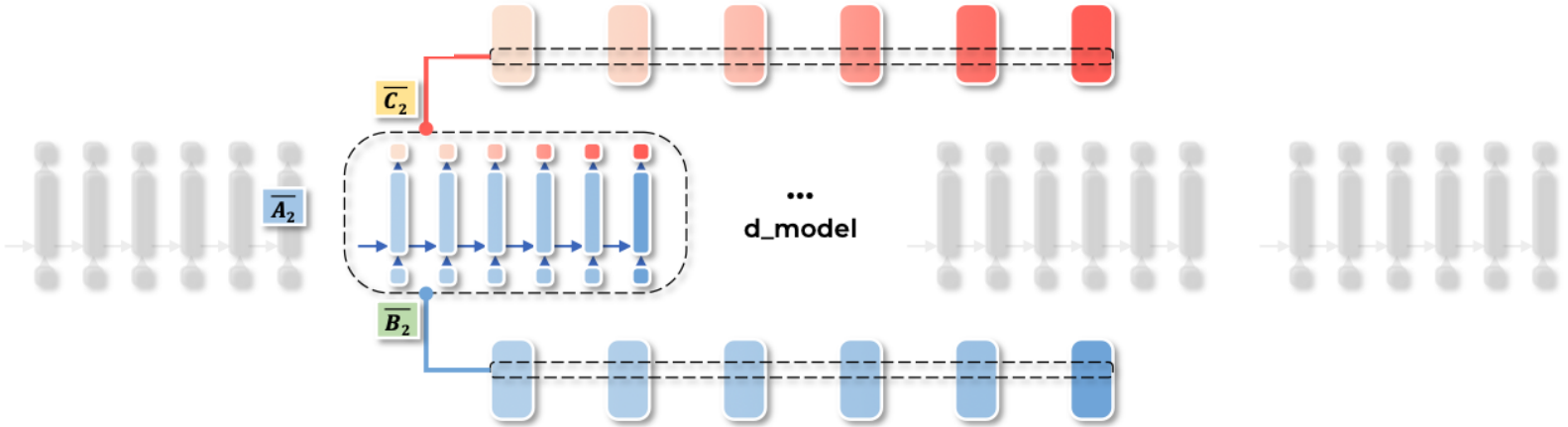
$$x_{t,1} \in \mathbb{R}^1$$
$$h_{t,1} \in \mathbb{R}^n$$
$$y_{t,1} \in \mathbb{R}^1$$

$$\overline{\mathrm{A}}_1 \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}}_1 \in \mathbb{R}^{n \times 1}$$
$$\mathrm{C}_1 \in \mathbb{R}^{1 \times n}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



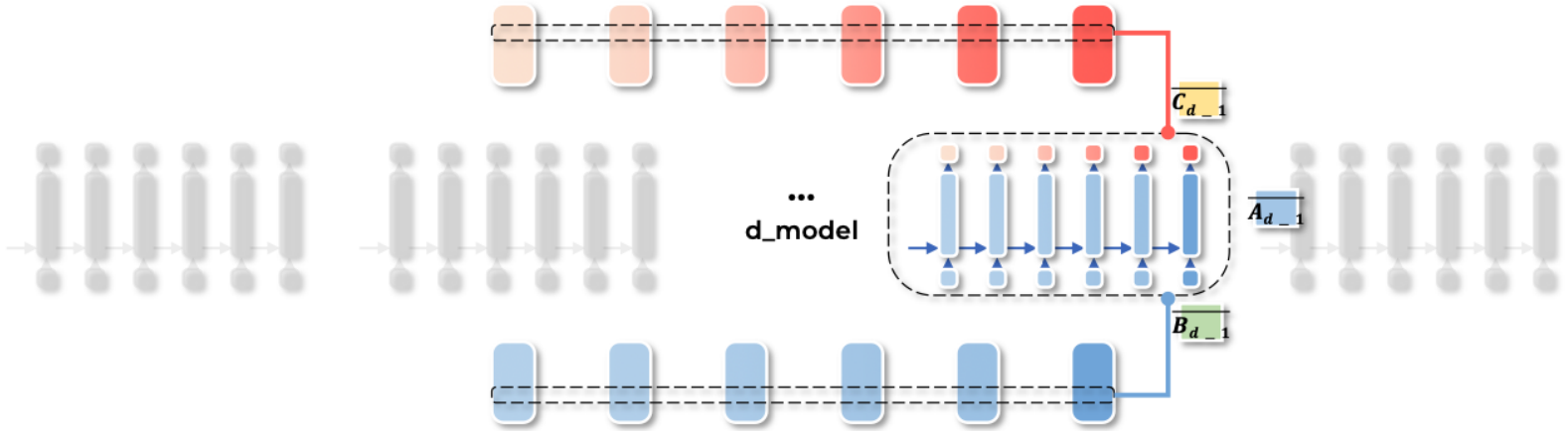
$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

$$x_{t,2} \in \mathbb{R}^1$$
$$h_{t,2} \in \mathbb{R}^n$$
$$y_{t,2} \in \mathbb{R}^1$$

$$\overline{\mathrm{A}}_2 \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}}_2 \in \mathbb{R}^{n \times 1}$$
$$\mathrm{C}_2 \in \mathbb{R}^{1 \times n}$$

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



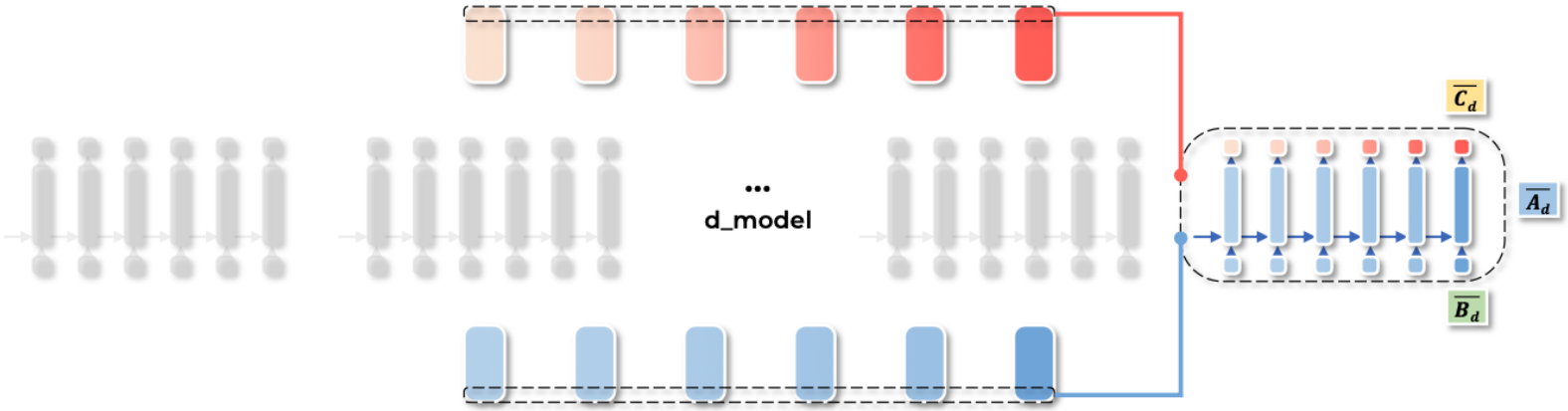
$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

$$x_{t,d\_model-1} \in \mathbb{R}^1$$
$$h_{t,d\_model-1} \in \mathbb{R}^n$$
$$y_{t,d\_model-1} \in \mathbb{R}^1$$

$$\overline{\mathrm{A}}_{d\_model-1} \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}}_{d\_model-1} \in \mathbb{R}^{n \times 1}$$
$$\mathrm{C}_{d\_model-1} \in \mathbb{R}^{1 \times n}$$

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

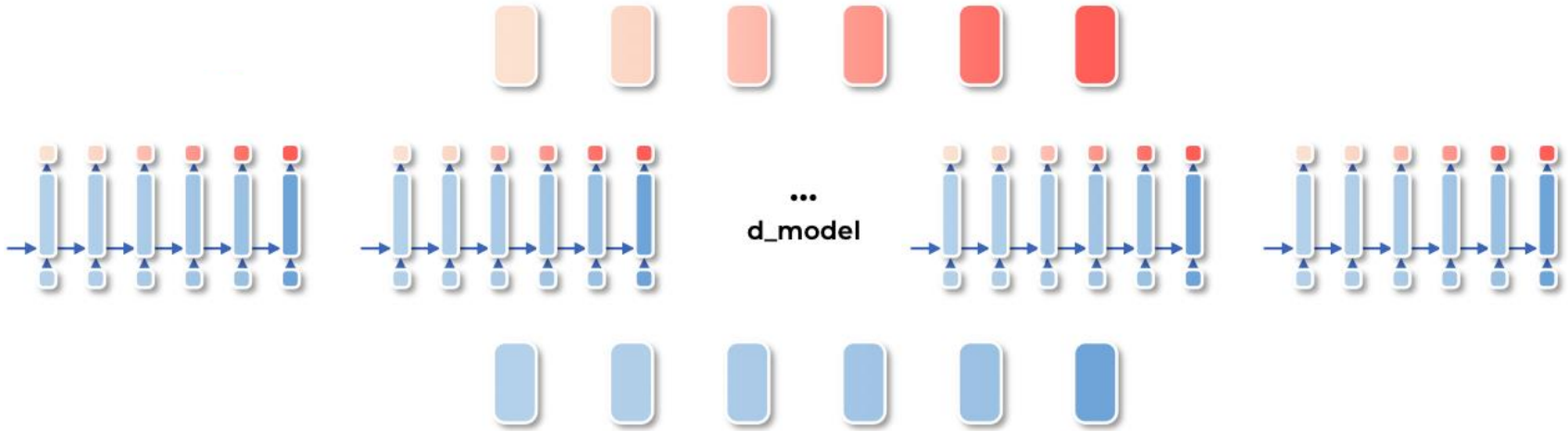
$$x_{t,d\_model} \in \mathbb{R}^1$$
$$h_{t,d\_model} \in \mathbb{R}^n$$
$$y_{t,d\_model} \in \mathbb{R}^1$$

$$\overline{\mathrm{A}}_{d\_model} \in \mathbb{R}^{n \times n}$$
$$\overline{\mathrm{B}}_{d\_model} \in \mathbb{R}^{n \times 1}$$
$$\mathrm{C}_{d\_model} \in \mathbb{R}^{1 \times n}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

Background

- State Space Model
  - Linear State Space Layer(LSSL): Dimension



$$h_t = \overline{\mathrm{A}} h_{t-1} + \overline{\mathrm{B}} x_t$$
$$y_t = \mathrm{C} h_t$$

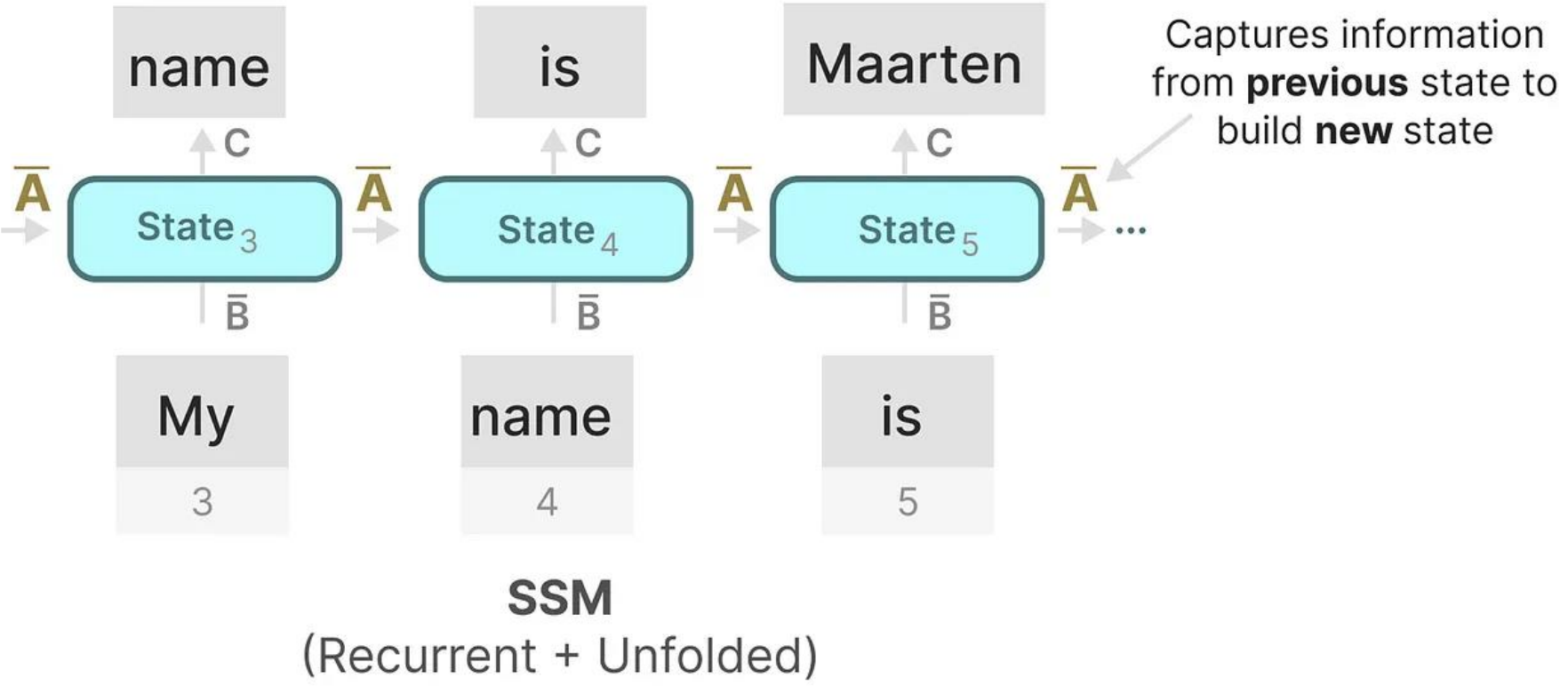
$$x_t \in \mathbb{R}^{d\_model}$$
$$h_t \in \mathbb{R}^{n}$$
$$y_t \in \mathbb{R}^{d\_model}$$

$$\overline{\mathrm{A}} \in \mathbb{R}^{n \times n \times d\_model}$$
$$\overline{\mathrm{B}} \in \mathbb{R}^{n \times 1 \times d\_model}$$
$$C \in \mathbb{R}^{1 \times n \times d\_model}$$

https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

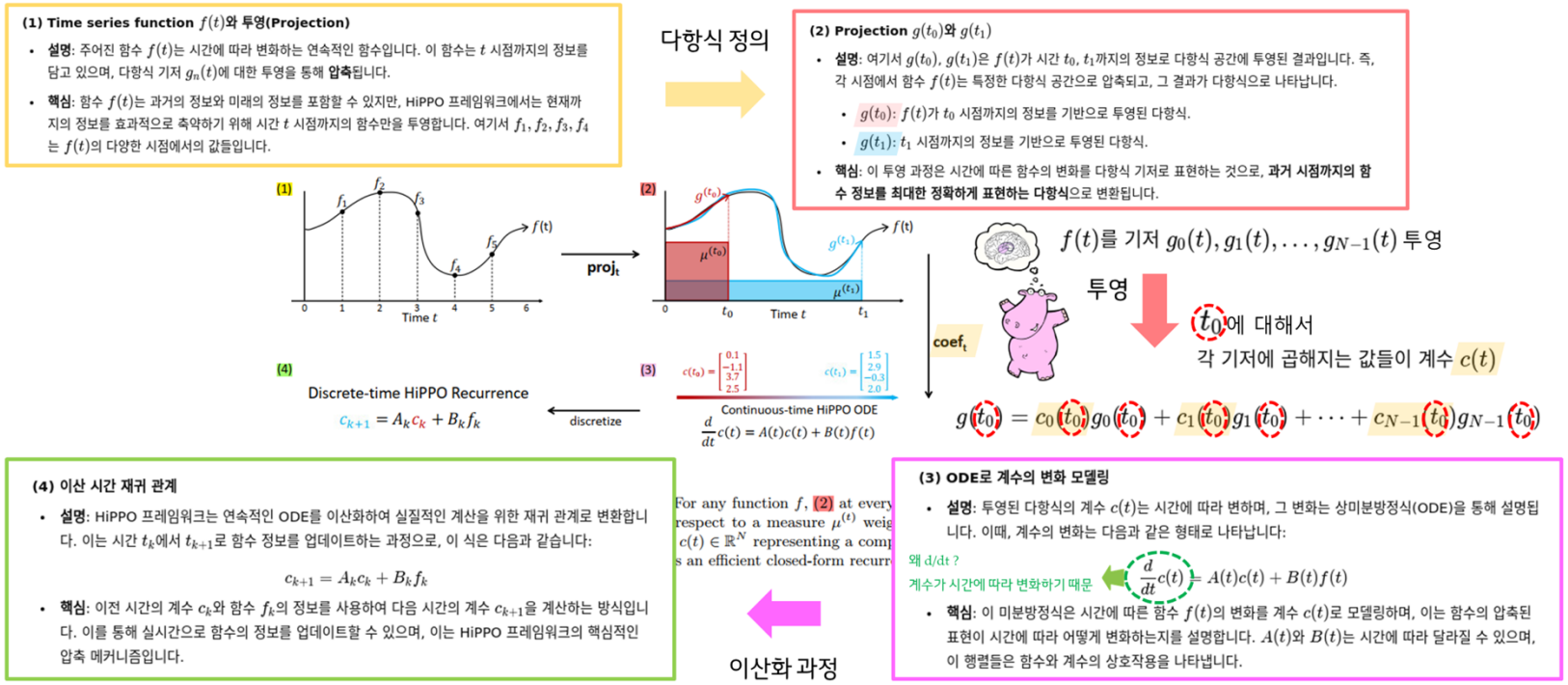
Background

- State Space Model
  - HiPPO: High-order Polynomial Projection Operators
    - A matrix
      - Update memory
      - Unstable gradient



Background

- State Space Model
  - HiPPO: High-order Polynomial Projection Operators
    - Limited memory horizon
    - Vanishing gradients



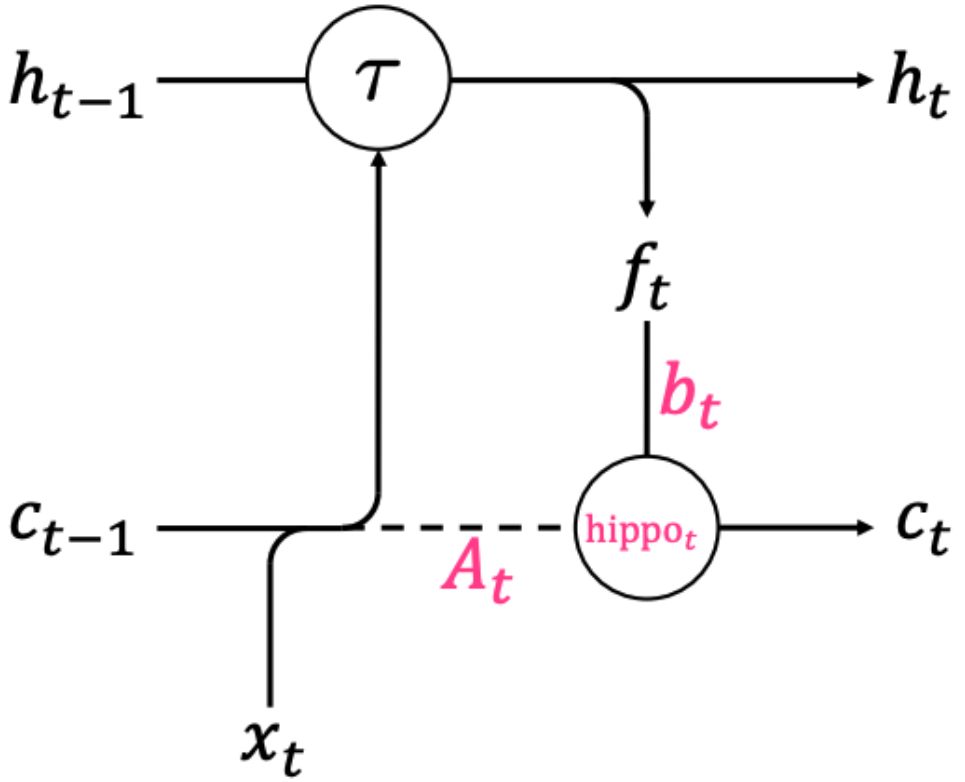
HiPPO: Recurrent Memory with Optimal Polynomial Projections
https://velog.io/@euisuk-chung/Structured-State-Space-Models-for-Deep-Sequence-Modeling

Background

- State Space Model
  - HiPPO: High-order Polynomial Projection Operators

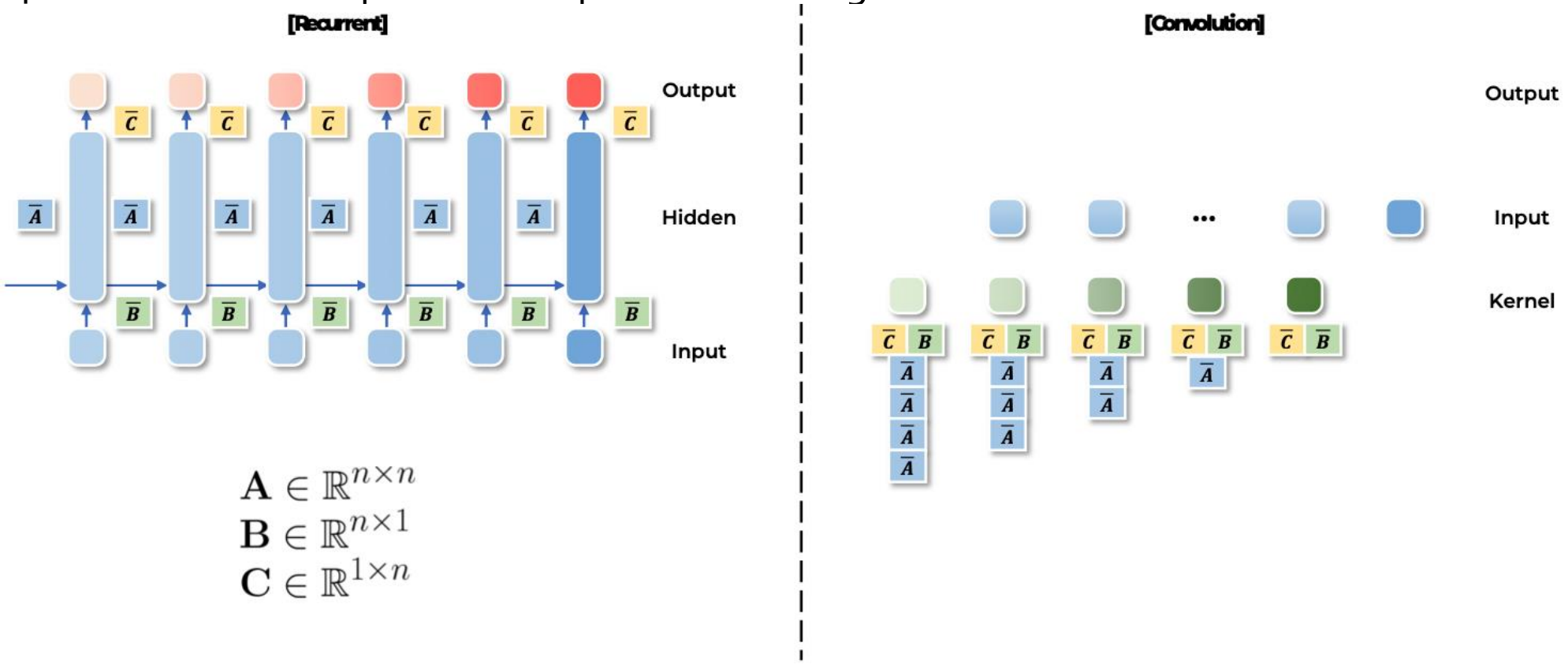
$$A_{nk} = \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & if\, n > k \\ n+1 & if\, n = k, \\ 0 & if\, n < k \end{cases}$$

$$B_n = (2n+1)^{\frac{1}{2}}$$



Background

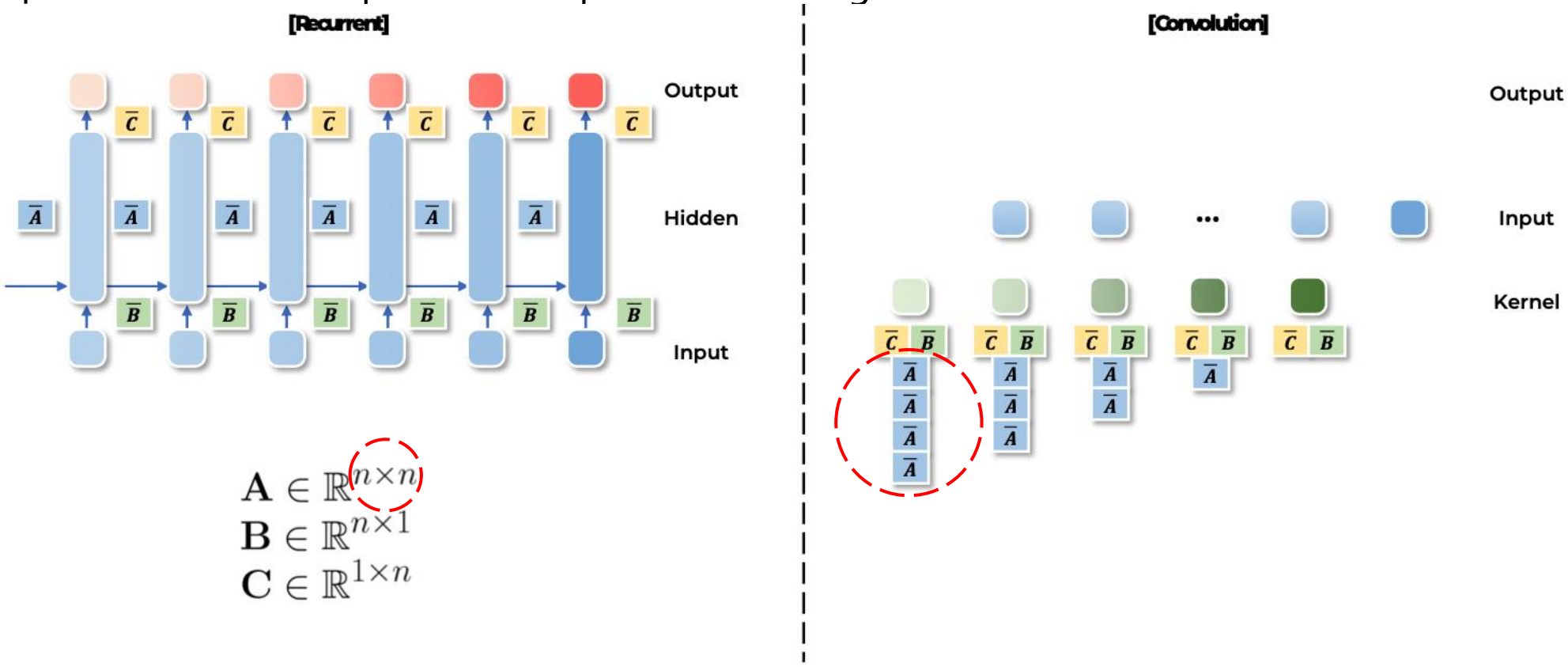
- State Space Model
  - S4: Structured State Spaces for Sequence modelling



https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s

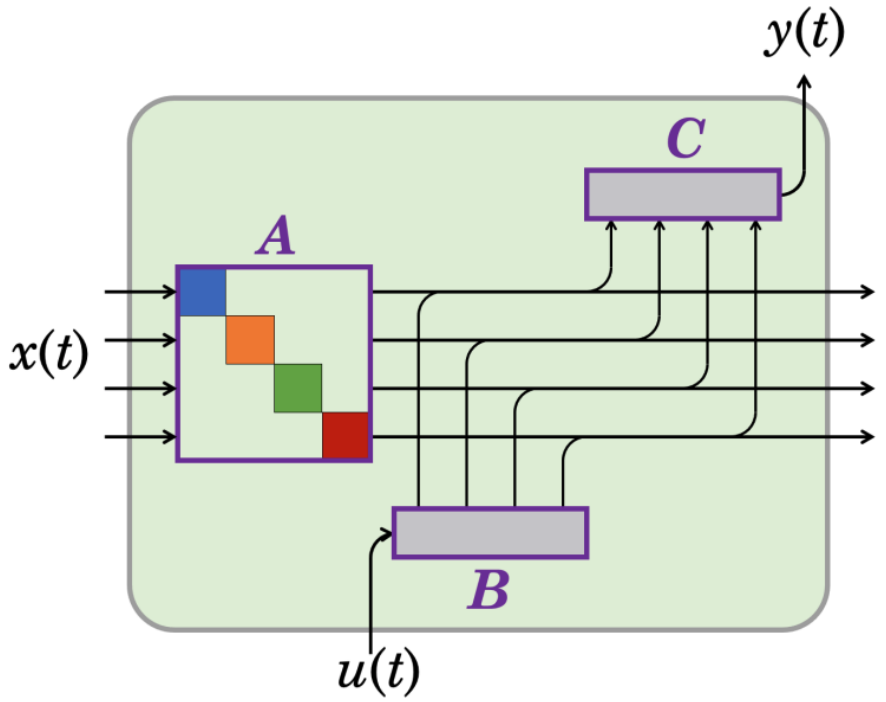
Background

- State Space Model
  - S4: Structured State Spaces for Sequence modelling



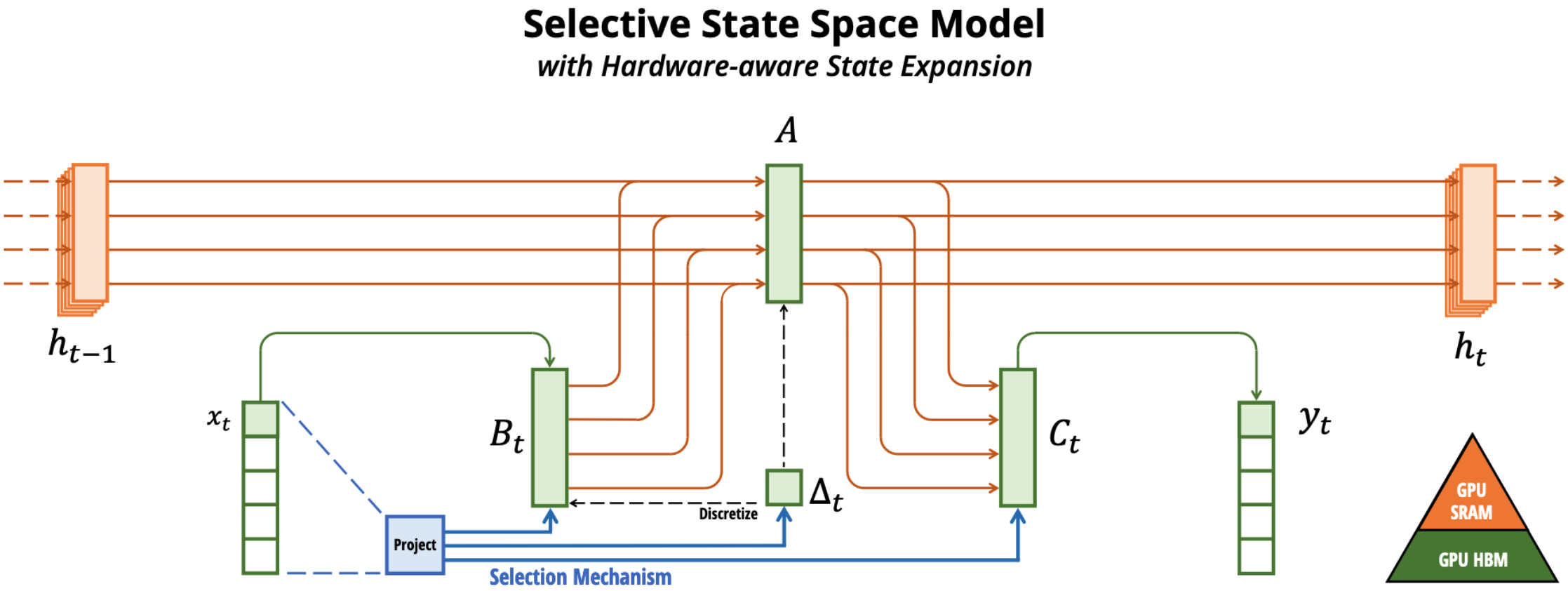
Background

- State Space Model
  - S4: Structured State Spaces



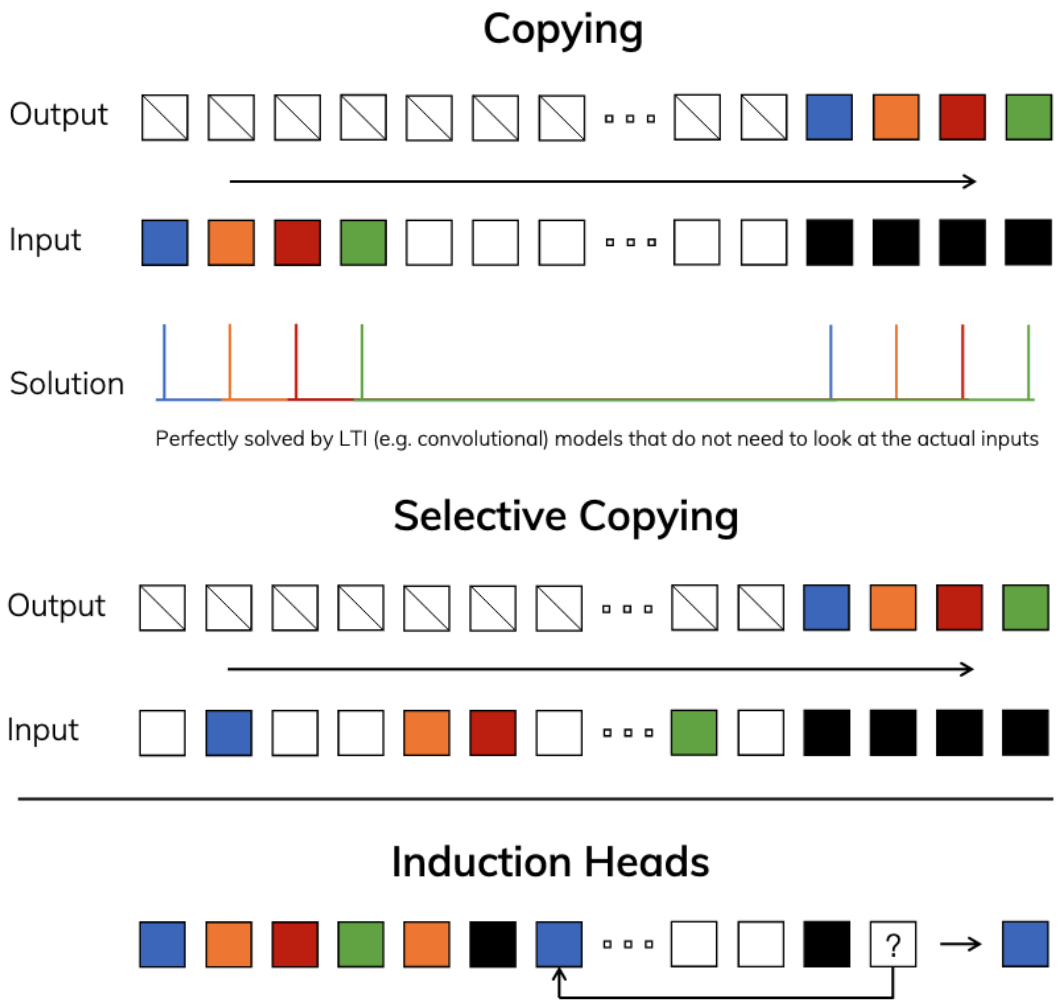
S4D Recurrent View

Mamba: Selective State Space Models



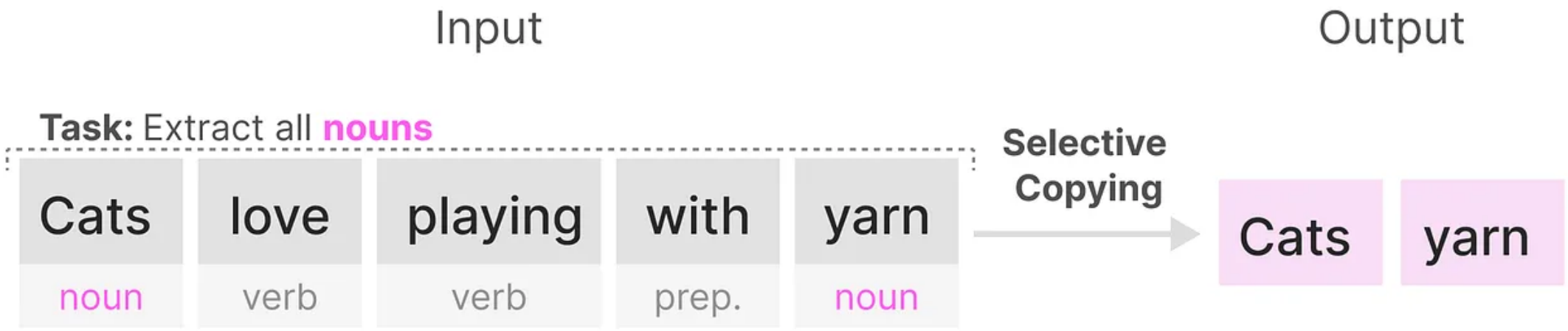
Mamba: Selective State Space Models

- Proposed Methods
  - Selection Mechanism
    - Selective Copying
    - Induction Heads
  - Hardware-aware Algorithm
    - Selective Scan



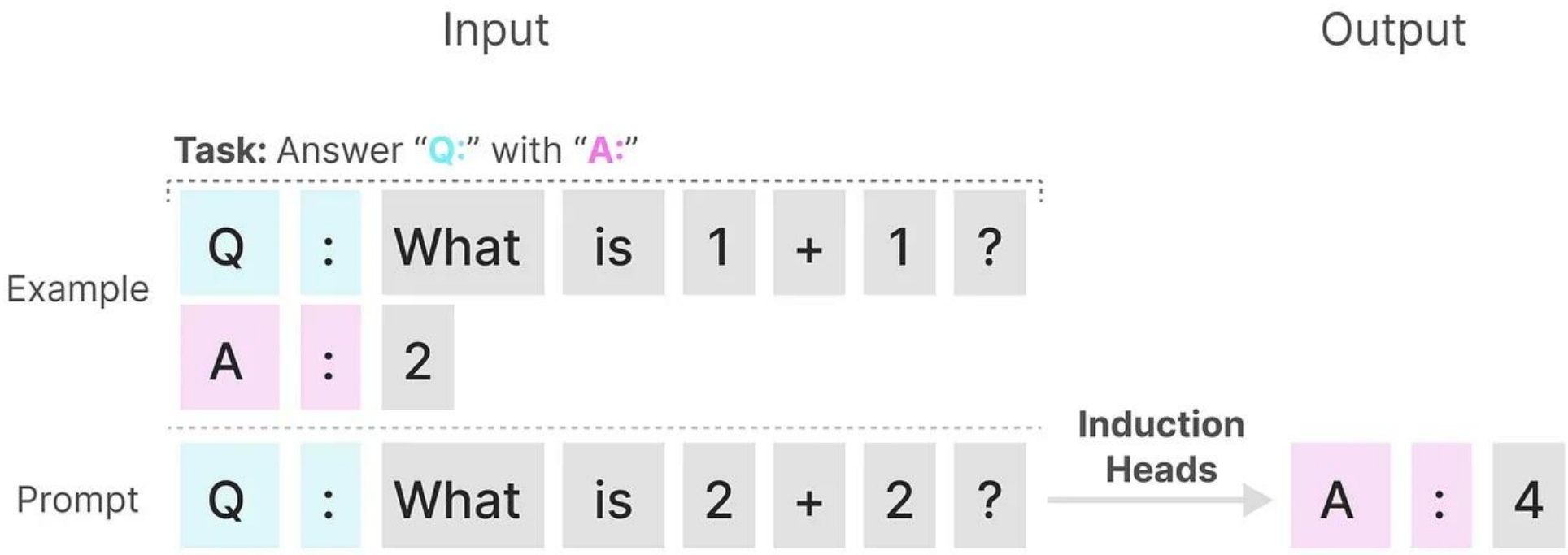
Mamba: Selective State Space Models

- Selection Mechanism
  - Selective Copying



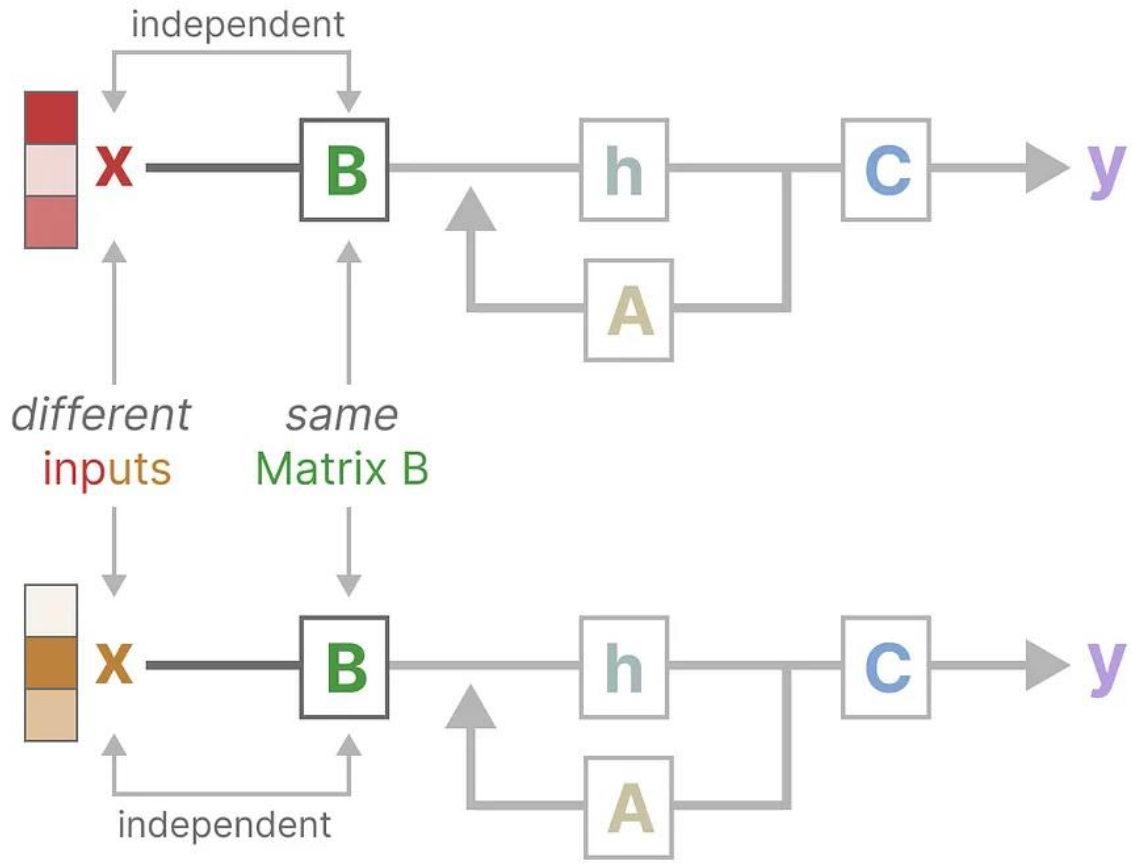
Mamba: Selective State Space Models

- Selection Mechanism
  - Induction Heads



Mamba: Selective State Space Models

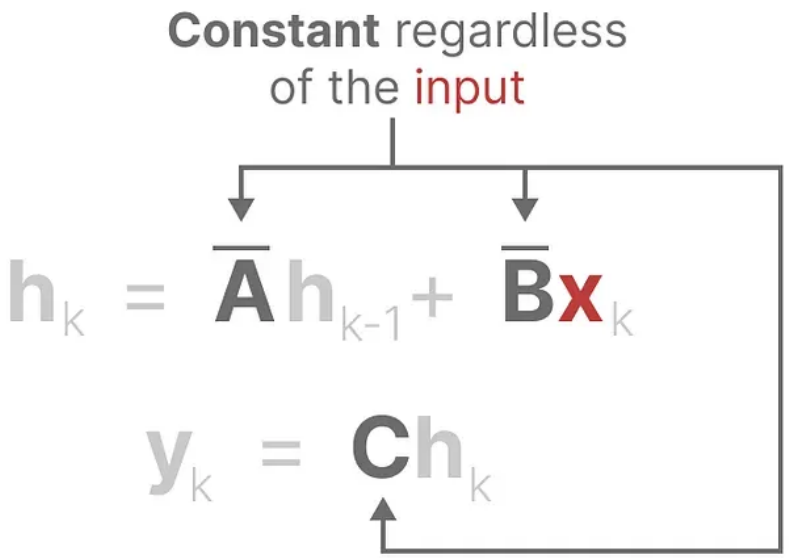
- Selection Mechanism
  - Linear Time Invarient



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Mamba: Selective State Space Models

- Selection Mechanism
  - Linear Time Invarient



Mamba: Selective State Space Models

- Selection Mechanism
  - SSM + Selection + Scan (S6)

Algorithm 1 SSM (S4)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
3: $\boldsymbol{C} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
4: $\Delta : (\mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter})$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

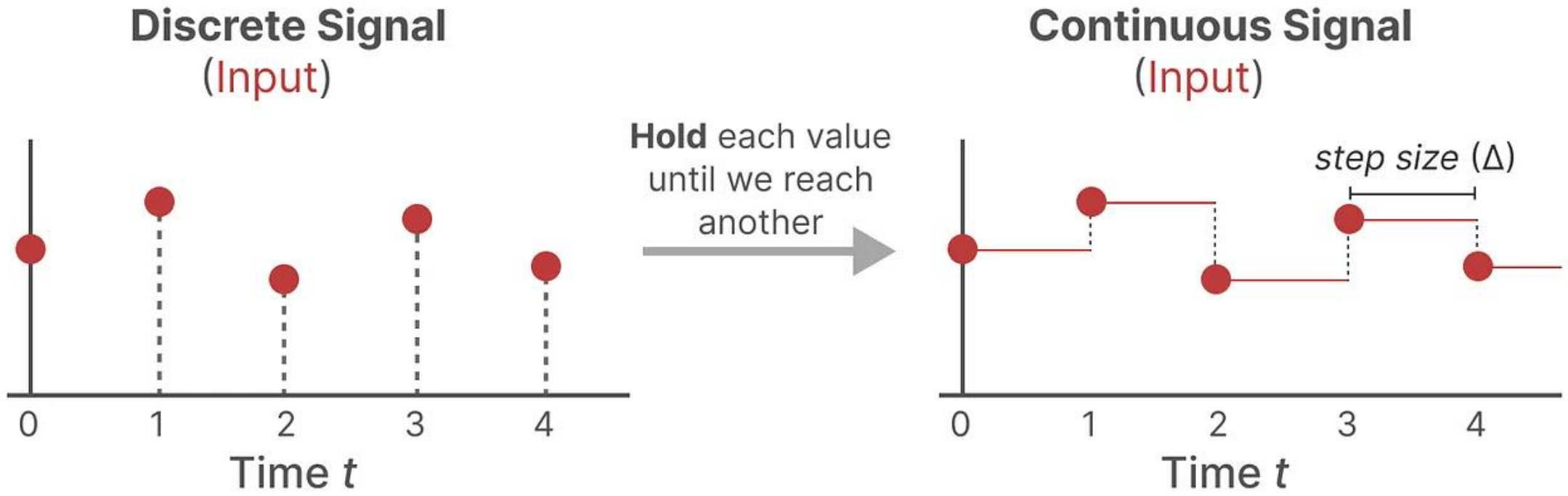
Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_B(x)$
3: $\boldsymbol{C} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_C(x)$
4: $\Delta : (\mathsf{B}, \mathsf{L}, \mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{B}, \mathsf{L}, \mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

$$s_B(x) = \text{Linear}_N(x)$$
$$s_C(x) = \text{Linear}_N(x)$$
$$s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x))$$

Mamba: Selective State Space Models

- State Space Model
  - Discretization: Zero-Order Hold



Mamba: Selective State Space Models

- State Space Model
  - Discretization: Zero-Order Hold

Discretized matrix **A**

$$\overline{\mathbf{A}} = \exp(\Delta \mathbf{A})$$

Discretized matrix **B**

$$\overline{\mathbf{B}} = (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - I) \cdot \Delta \mathbf{B}$$

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Mamba: Selective State Space Models

- Selection Mechanism
  - SSM + Selection + Scan (S6)

Algorithm 1 SSM (S4)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
3: $\boldsymbol{C} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
4: $\Delta : (\mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter})$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_B(x)$
3: $\boldsymbol{C} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_C(x)$
4: $\Delta : (\mathsf{B}, \mathsf{L}, \mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{B}, \mathsf{L}, \mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

$$s_B(x) = \text{Linear}_N(x)$$
$$s_C(x) = \text{Linear}_N(x)$$
$$s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x))$$

Mamba: Selective State Space Models

- Selection Mechanism
  - SSM + Selection + Scan (S6)

Discretized matrix **A**

$$\overline{\mathbf{A}} = \exp(\Delta \mathbf{A})$$

Discretized matrix **B**

$$\overline{\mathbf{B}} = (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - I) \cdot \Delta \mathbf{B}$$

  - $\Delta \to 0$: persists the state and ignores the current input
  - $\Delta \to \infty$: resets the state and focuses on the current input

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

Mamba: Selective State Space Models

- Selection Mechanism
  - SSM + Selection + Scan (S6)

Discretized matrix **A**

$$\overline{\mathbf{A}} = \exp(\Delta \mathbf{A})$$

Discretized matrix **B**

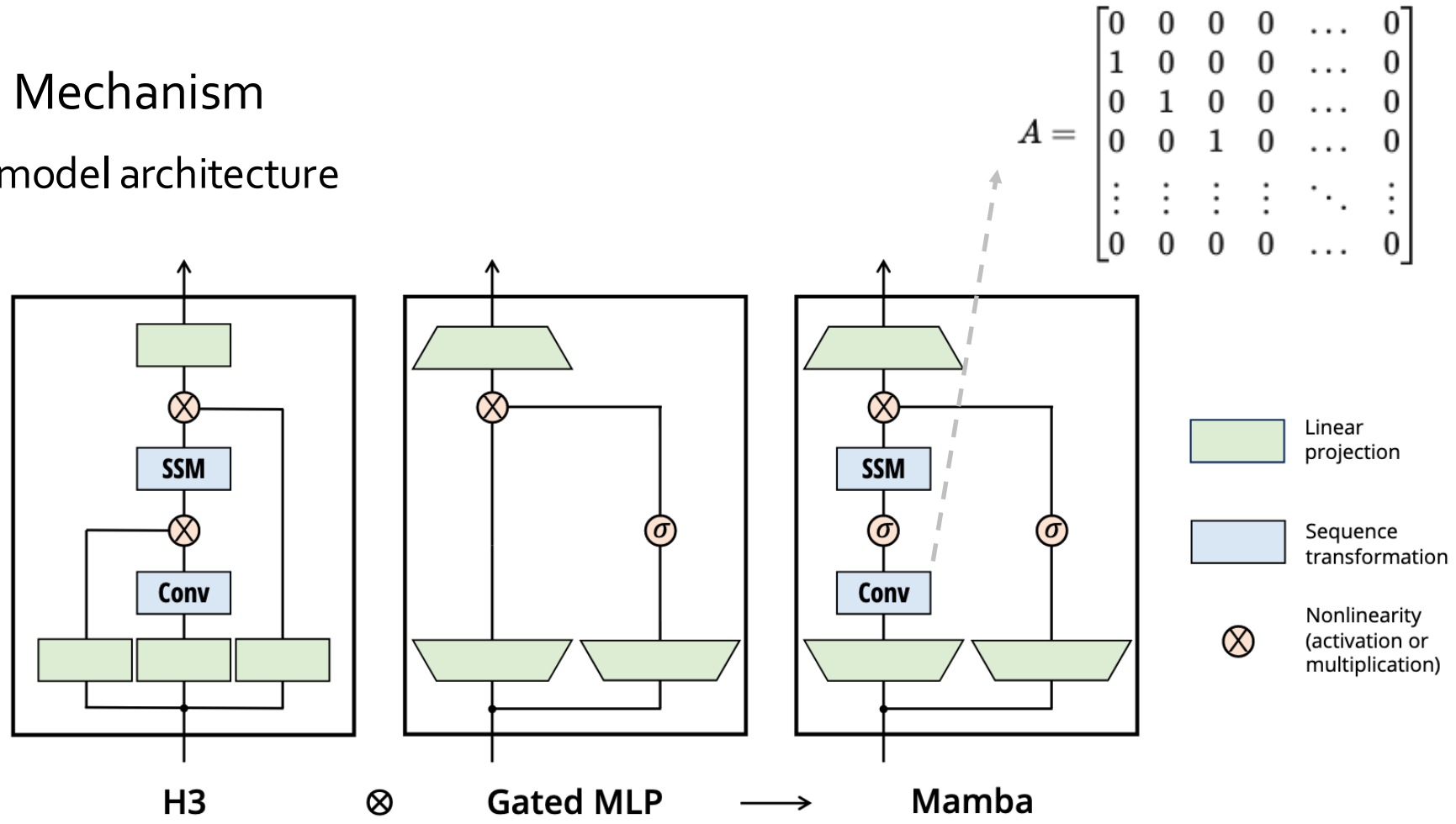
$$\overline{\mathbf{B}} = (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - I) \cdot \Delta \mathbf{B}$$

  - $\Delta \to 0$: persists the state and ignores the current input
  - $\Delta \to \infty$: resets the state and focuses on the current input

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state

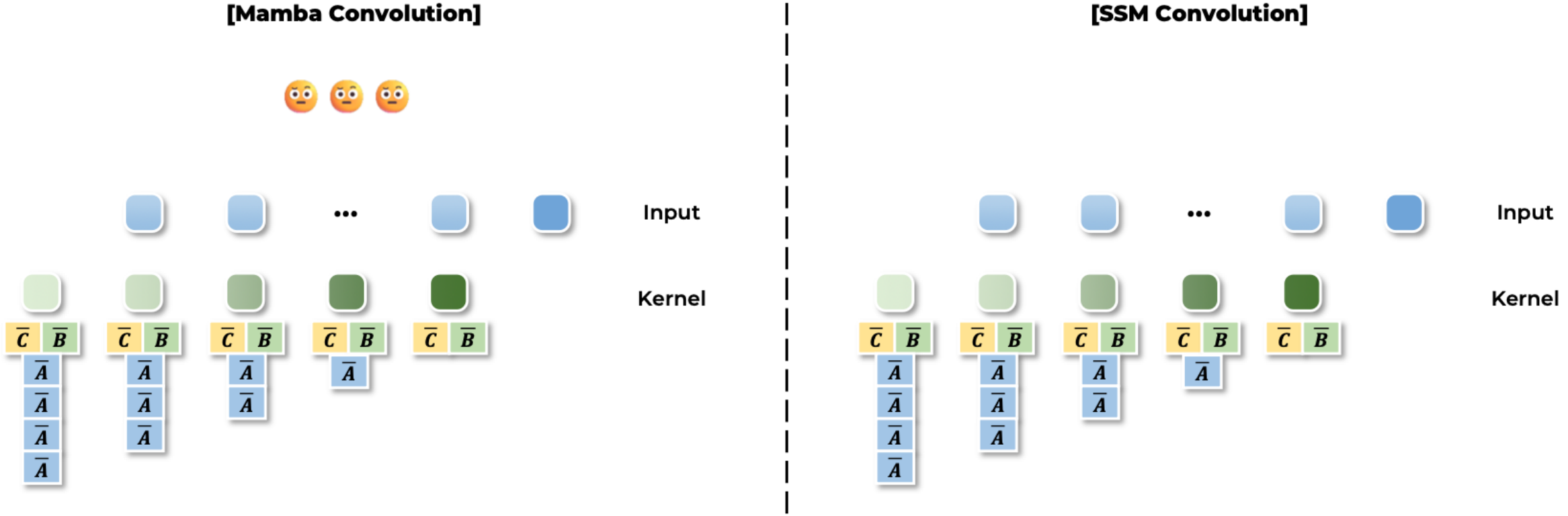
Mamba: Selective State Space Models

- Selection Mechanism
  - Whole model architecture



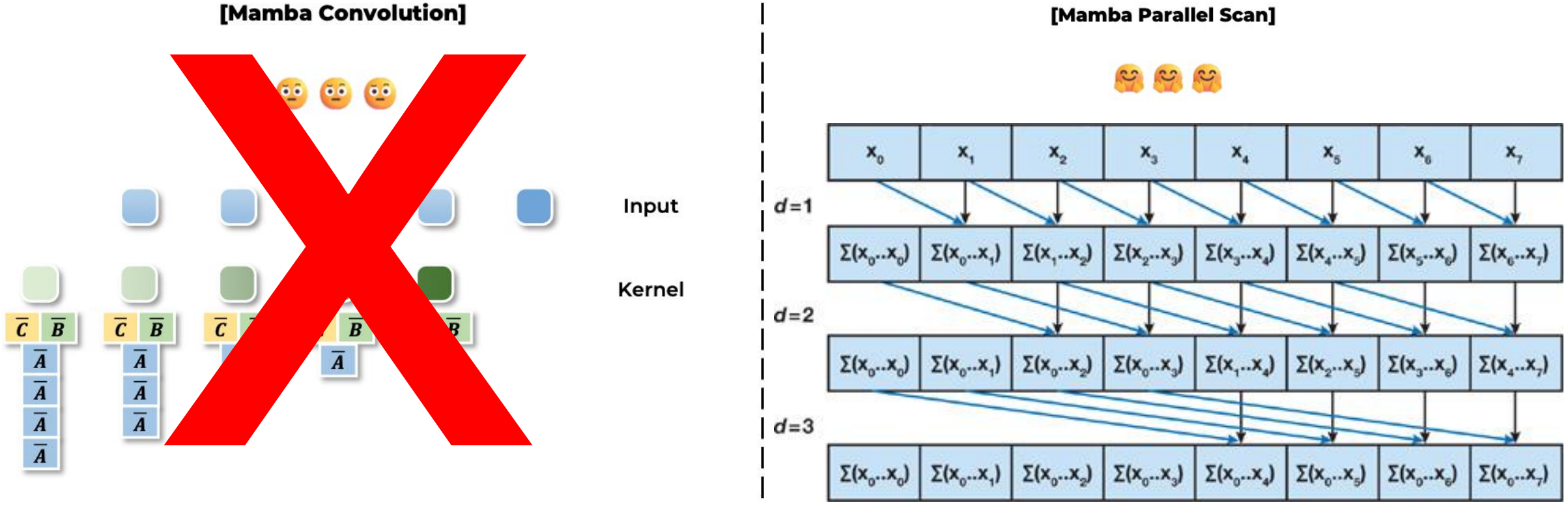
Mamba: Selective State Space Models

- Hardware-aware Algorithm
  - Is still can Training Parallelization?



Mamba: Selective State Space Models

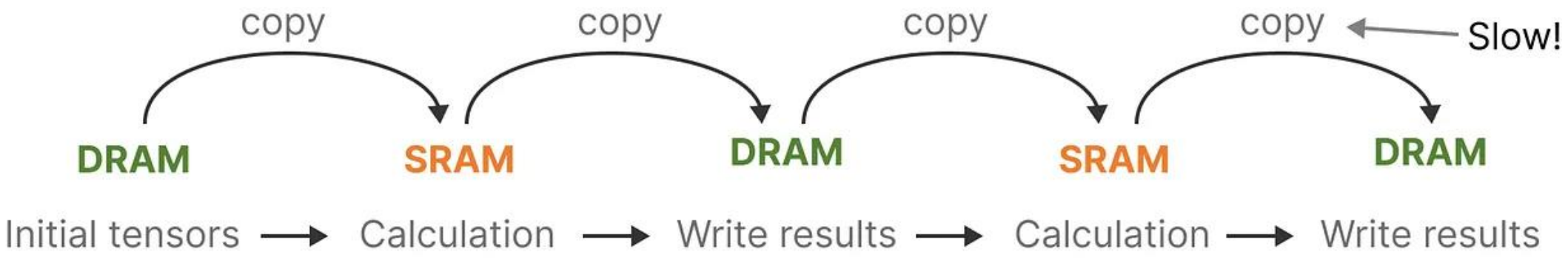
- Hardware-aware Algorithm
  - Parallel Scan



https://youtu.be/JjxBNBzDbNk?si=-uXfWPjmshdfJd2s
https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda

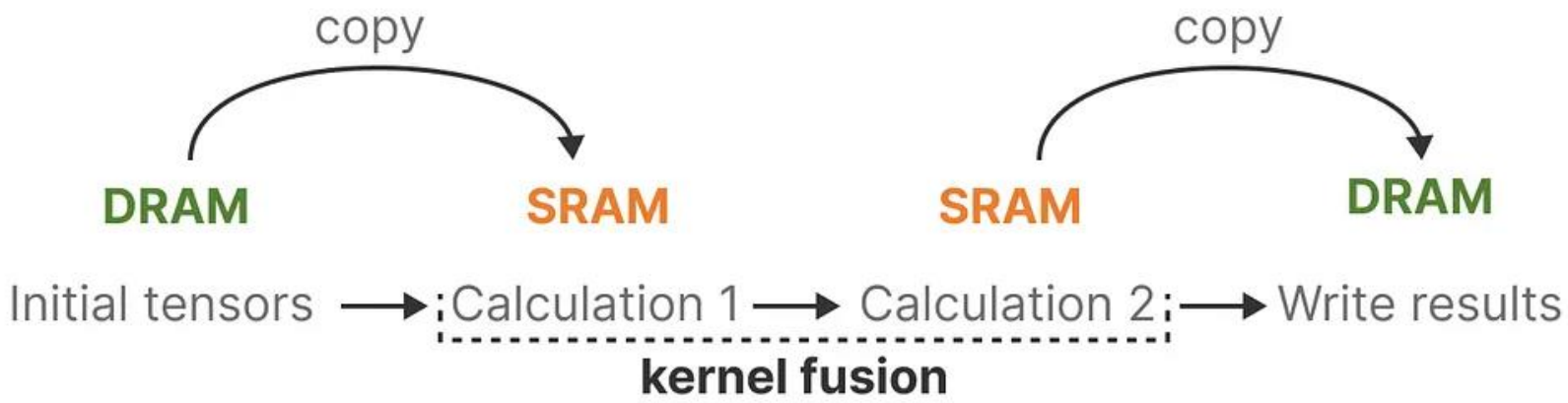
Mamba: Selective State Space Models

- Hardware-aware Algorithm
  - Kernel Fusion
  - SSM / D N -> Mamba / B L D N vs Transformer / B L D



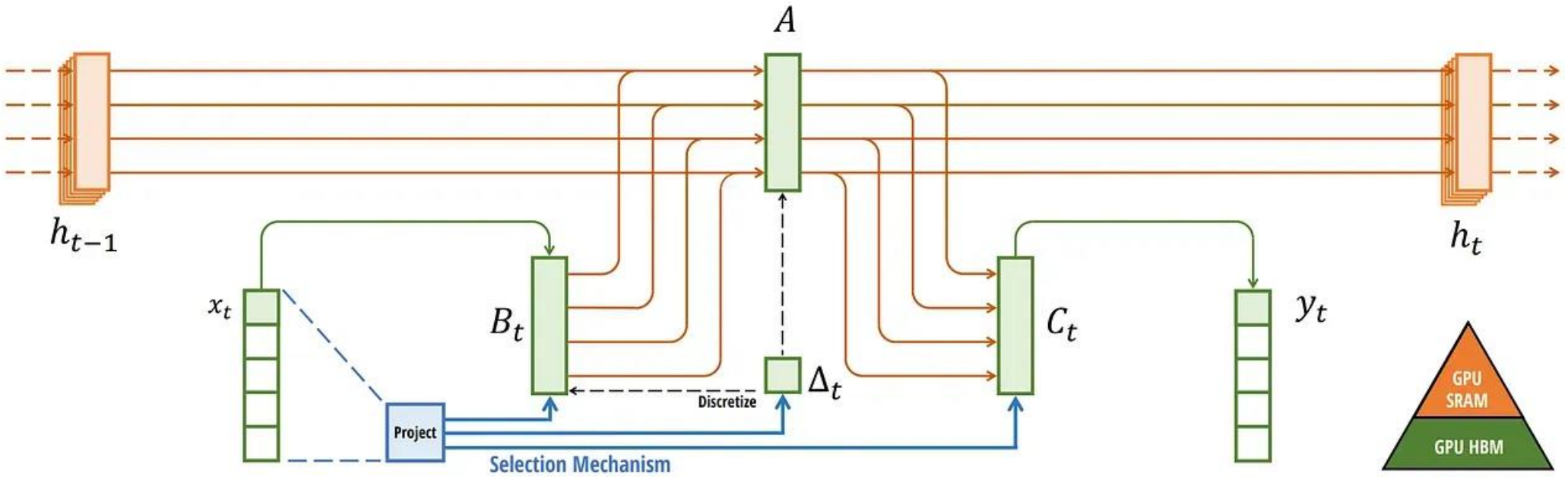
Mamba: Selective State Space Models

- Hardware-aware Algorithm
  - Kernel Fusion



Mamba: Selective State Space Models

- Hardware-aware Algorithm
  - Kernel Fusion



Experiments

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Table 1: (**Selective Copying.**) Accuracy for combinations of architectures and inner sequence layers.

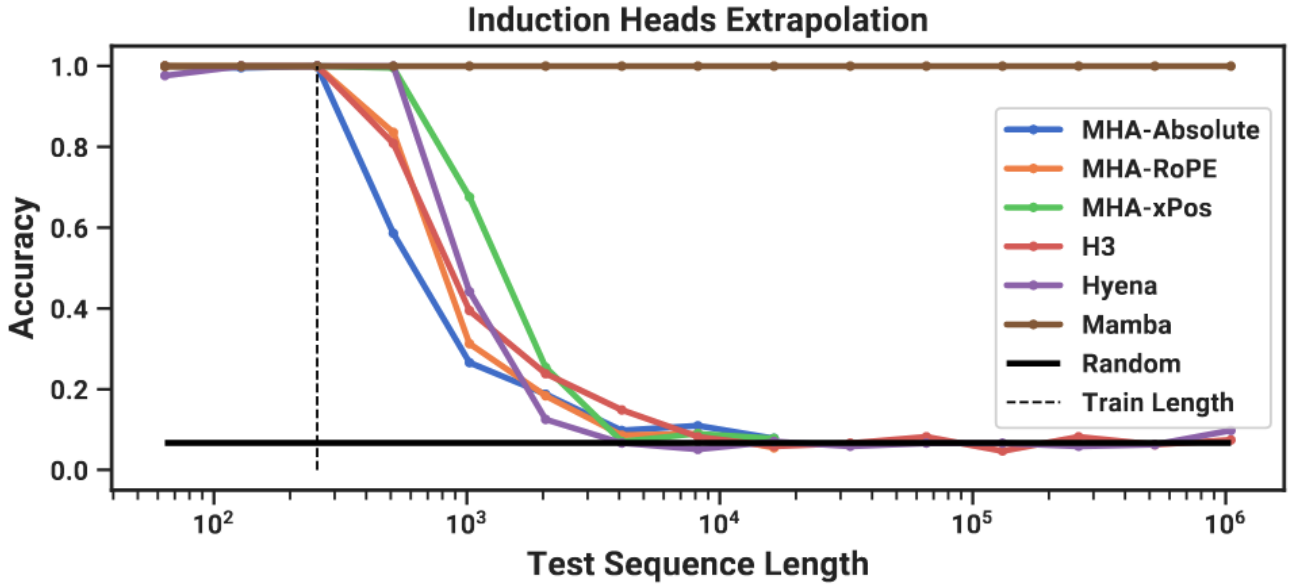


Table 2: (**Induction Heads.**) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.

Experiments

Table 11: (**Induction heads.**) Models are trained on sequence length $2^8 = 256$, and tested on various sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. ✓ denotes perfect generalization accuracy, while ✗ denotes out of memory.

Model	Params	Test Accuracy (%) at Sequence Length														
		2^6	2^7	$\mathbf{2^8}$	2^9	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	2^{17}	2^{18}	2^{19}	2^{20}
MHA-Abs	137K	✓	99.6	100.0	58.6	26.6	18.8	9.8	10.9	7.8	✗	✗	✗	✗	✗	✗
MHA-RoPE	137K	✓	✓	100.0	83.6	31.3	18.4	8.6	9.0	5.5	✗	✗	✗	✗	✗	✗
MHA-xPos	137K	✓	✓	100.0	99.6	67.6	25.4	7.0	9.0	7.8	✗	✗	✗	✗	✗	✗
H3	153K	✓	✓	100.0	80.9	39.5	23.8	14.8	8.2	5.9	6.6	8.2	4.7	8.2	6.3	7.4
Hyena	69M*	97.7	✓	100.0	✓	44.1	12.5	6.6	5.1	7.0	5.9	6.6	6.6	5.9	6.3	9.8
Mamba	74K	✓	✓	100.0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

* Most of the parameters are in learnable positional encodings.

Experiments

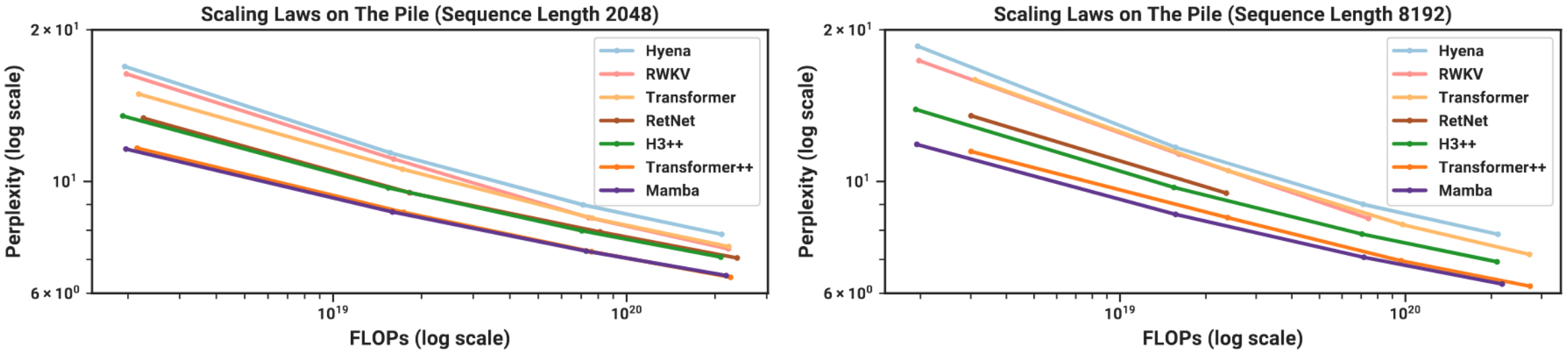


Figure 4: (**Scaling Laws.**) Models of size $\approx 125M$ to $\approx 1.3B$ parameters, trained on the Pile. Mamba scales better than all other attention-free models and is the first to match the performance of a very strong "Transformer++" recipe that has now become standard, particularly as the sequence length grows.

Experiments

Table 3: (**Zero-shot Evaluations**.) Best results for each size in bold. We compare against open source LMs with various tokenizers, trained for up to 300B tokens. Pile refers to the validation split, comparing only against models trained on the same dataset and tokenizer (GPT-NeoX-20B). For each model size, Mamba is best-in-class on every single evaluation result, and generally matches baselines at twice the model size.

Model	Token.	Pile ppl ↓	LAMBADA ppl ↓	LAMBADA acc ↑	HellaSwag acc ↑	PIQA acc ↑	Arc-E acc ↑	Arc-C acc ↑	WinoGrande acc ↑	Average acc ↑
Hybrid H3-130M	GPT2	—	89.48	25.77	31.7	64.2	44.4	24.2	50.6	40.1
Pythia-160M	NeoX	29.64	38.10	33.0	30.2	61.4	43.2	24.1	**51.9**	40.6
Mamba-130M	NeoX	**10.56**	**16.07**	**44.3**	**35.3**	**64.5**	**48.0**	**24.3**	**51.9**	**44.7**
Hybrid H3-360M	GPT2	—	12.58	48.0	41.5	68.1	51.4	24.7	54.1	48.0
Pythia-410M	NeoX	9.95	10.84	51.4	40.6	66.9	52.1	24.6	53.8	48.2
Mamba-370M	NeoX	**8.28**	**8.14**	**55.6**	**46.5**	**69.5**	**55.1**	**28.0**	**55.3**	**50.0**
Pythia-1B	NeoX	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	51.9
Mamba-790M	NeoX	**7.33**	**6.02**	**62.7**	**55.1**	**72.1**	**61.2**	**29.5**	**56.1**	**57.1**
GPT-Neo 1.3B	GPT2	—	7.50	57.2	48.9	71.1	56.2	25.9	54.9	52.4
Hybrid H3-1.3B	GPT2	—	11.25	49.6	52.6	71.3	59.2	28.1	56.9	53.0
OPT-1.3B	OPT	—	6.64	58.0	53.7	72.4	56.7	29.6	59.5	55.0
Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	55.2
RWKV-1.5B	NeoX	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	54.3
Mamba-1.4B	NeoX	**6.80**	**5.04**	**64.9**	**59.1**	**74.2**	**65.5**	**32.8**	**61.5**	**59.7**
GPT-Neo 2.7B	GPT2	—	5.63	62.2	55.8	72.1	61.1	30.2	57.6	56.5
Hybrid H3-2.7B	GPT2	—	7.92	55.7	59.7	73.3	65.6	32.3	61.4	58.0
OPT-2.7B	OPT	—	5.12	63.6	60.6	74.8	60.8	31.3	61.0	58.7
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	59.1
RWKV-3B	NeoX	7.00	5.24	63.9	59.6	73.7	67.8	33.1	59.6	59.6
Mamba-2.8B	NeoX	**6.22**	**4.23**	**69.2**	**66.1**	**75.2**	**69.7**	**36.3**	**63.5**	**63.3**
GPT-J-6B	GPT2	–	4.10	68.3	66.3	75.4	67.0	36.6	64.1	63.0
OPT-6.7B	OPT	–	4.25	67.7	67.2	76.3	65.6	34.9	65.5	62.9
Pythia-6.9B	NeoX	6.51	4.45	67.1	64.0	75.2	67.3	35.5	61.3	61.7
RWKV-7.4B	NeoX	6.31	4.38	67.2	65.5	76.1	67.8	37.5	61.0	62.5

Experiments

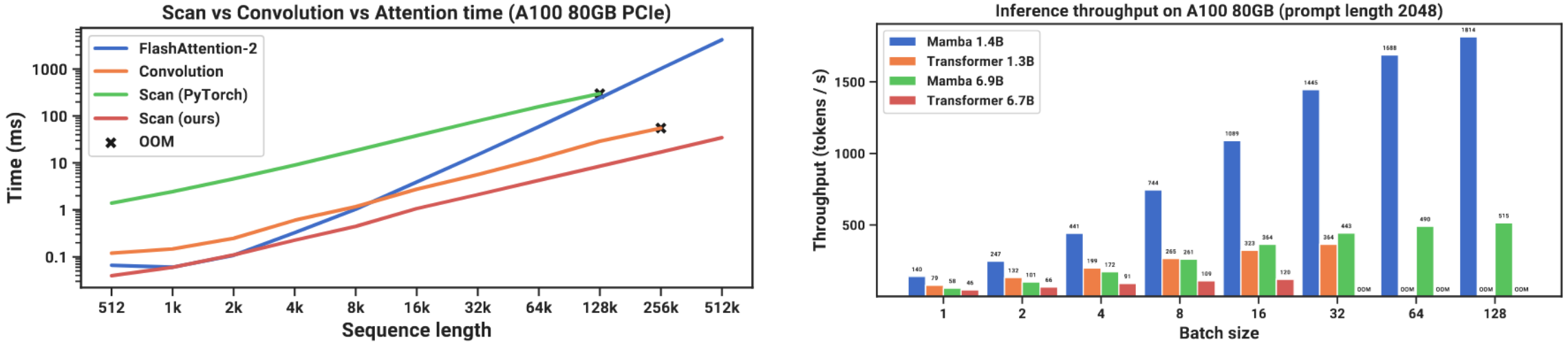


Figure 8: (**Efficiency Benchmarks**.) (*Left*) Training: our efficient scan is 40× faster than a standard implementation. (*Right*) Inference: as a recurrent model, Mamba can achieve 5× higher throughput than Transformers.


Thank You!